



BCSE498J- Project - II

AI POWERED 2D ANIMATION GENERATOR

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE3217 GOLLA PAVAN SHIVA

Under the Supervision of

Prof. Meenakshi S P

Assistant Professor Sr. Grade 2

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled “AI POWERED 2D ANIMATION GENERATOR” submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Meenakshi S P, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

GOLLA PAVAN SHIVA (22BCE3217)

Date :

CERTIFICATE

This is to certify that the project entitled “AI POWERED 2D ANIMATION GENERATOR” submitted by GOLLA PAVAN SHIVA (22BCE3217), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date :

Internal Examiner

External Examiner

Head of the Department

ACKNOWLEDGEMENT

I GOLLA PAVAN SHIVA (22BCE3217) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Meenakshi S P, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

Executive Summary	ii
List of Tables	iii
List of Figures	iv
List of Abbreviations	v
List of Symbols	vi
1 INTRODUCTION	1
1.1 Background	1
1.1.1 Current Animation Systems and Their Limitations	2
1.1.2 Proposed System Description	2
1.2 Motivation	2
1.3 Scope of the project	3
2 PROJECT DESCRIPTION AND GOALS	5
2.1 Literature review	5
2.2 Research Gap	10
2.2.1 Integrated LLM-Animation Pipeline Deficiency	10
2.2.2 Intelligent Video Merging and Concatenation Gap	10
2.2.3 Limitations of Real-Time Collaborative Animation Editing . . .	11
2.2.4 Shortage of Domain-Specific Animation Code Generation . . .	11
2.3 Objectives	12
2.4 Problem Statement	13
2.5 Project Plan	14
2.5.1 Phase 1: Planning and Research (August 2025)	14
2.5.2 Phase 2: Requirement Analysis & Design (August – September 2025)	15
2.5.3 Phase 3: Development (September 2025 – February 2026) . . .	15
2.5.4 Phase 4: Testing and Verification (February – April 2026) . . .	17
2.5.5 Phase 5: Documentation (April 2026)	17

3	TECHNICAL SPECIFICATION	18
3.1	Requirements	18
3.1.1	Functional Requirements	18
3.1.2	Non-Functional Requirements	18
3.2	Feasibility Analysis	18
3.2.1	Technical Feasibility	18
3.2.2	Economic Feasibility	19
3.2.3	Socio-Economics	19
3.3	System Specification	19
3.3.1	Hardware Specification	19
3.3.2	Software Specification	19
3.4	IMPLEMENTATION DETAILS	20
3.4.1	Tables	20
3.4.2	Equations	22
3.4.3	Figures	22
3.4.4	Pseudo code / Algorithm	23
4	SYSTEM DESIGN	29
4.1	System Architecture	29
4.2	Detailed Design	30
4.2.1	DATA FLOW DIAGRAM	30
4.2.2	USE CASE DIAGRAM	31
4.2.3	CLASS DIAGRAM	31
4.2.4	SEQUENCE DIAGRAM	32
5	METHODOLOGY AND TESTING	33
5.1	Module 1: Frontend (ReactJS)	33
5.2	Module 2: Backend (FastAPI)	33
5.3	Module 3: AI Logic & Code Generation (LangGraph)	35
5.4	Module 4: Animation Rendering (Manim / Subprocess)	35
5.5	Module 5: Video Processing (MoviePy / FFmpeg)	36
5.6	Module 6: Cloud Storage (AWS S3)	36
5.7	Testing	36

5.7.1	1. Unit Testing	36
5.7.2	2. Integration Testing	36
5.7.3	3. End-to-End System Testing	37
5.7.4	4. Error Handling Testing	37
5.7.5	5. Security Testing	37
6	PROJECT IMPLEMENTATION	38
6.1	Step 1: Animation Generation	38
6.2	Step 2: Video Selection and Management	39
6.3	Step 3: Editing Your Animation	39
6.4	Step 4: Exporting Your Final Video	40
7	RESULTS AND DISCUSSION	42
8	CONCLUSION AND FURTHER ENHANCEMENTS	44
8.1	Conclusion	44
8.2	Future Improvements	44

EXECUTIVE SUMMARY

The AI-Powered Manim Animation Generator serves as an all-in-one solution delivered through a full-stack web application. It is capable of creating animations simply by users supplying their instructions in natural language. The main purpose of this work is to make the generation process so easy that a user wouldn't even have to write Manim animation scripts anymore. This would democratize the process not just for students but also for educators.

One of the major strengths of the suggested solution is that it taps into LLMs through the OpenAI API and LangChain to turn user input into legitimate Manim scripts. After that, the script is forwarded to the FastAPI server-side, which not only checks the input, but also deals with potential problems, creates the video from the script, and even manages the cases when video rendering fails.

Concerning the frontend part, the application has been built with React providing the functionality for users to enter their prompts, view the generated video, and download them. Moreover, for storing the video files in a cost-effective manner, the project brings in Amazon Web Services S3 and videos can be accessed through presigned URLs.

To summarize, this project shows how to effectively use artificial intelligence, automation and cloud services to develop a full-fledged application.

LIST OF TABLES

2.1	Literature Review of AI-Based Animation Systems	6
2.2	Literature Review of AI-Based Animation Systems	7
2.3	Literature Review of AI-Based Animation Systems	8
2.4	Literature Review of AI-Based Animation Systems	9
3.1	Hardware Requirements	19
3.2	Technology Stack Used	21
3.3	System Workflow	21
3.4	Performance Analysis	22

LIST OF FIGURES

3.1	System Architecture	22
3.2	Execution Time Complexity	23
3.3	Performance Metrics	23
4.1	Data flow diagram	30
4.2	Use Case diagram	31
4.3	Class diagram	31
4.4	Sequence diagram	32
5.1	langgraph pipeline	35
6.1	Landing Page	38
6.2	Selecting Videos	39
6.3	Video Editing Page	40
6.4	Video Preview And Export	41

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
API	Application Programming Interface
UI	User Interface
UX	User Experience
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
REST	Representational State Transfer
AWS	Amazon Web Services
S3	Simple Storage Service
CPU	Central Processing Unit
GPU	Graphics Processing Unit
TTS	Text-to-Speech
FFmpeg	Fast Forward Moving Picture Experts Group
MoviePy	Python Library for Video Editing
JWT	JSON Web Token
CORS	Cross-Origin Resource Sharing
NLP	Natural Language Processing

SYMBOLS AND NOTATIONS

$f(x)$	Function representing system transformation
Prompt	User input given in natural language
LLM	Large Language Model used for code generation
GeneratedCode	Manim Python code generated by AI
OutputVideo	Video generated after rendering process
Renderer	Manim rendering engine
RefinedPrompt	Modified prompt after error correction
RetryCount	Number of attempts to regenerate code
State	Workflow state in LangGraph pipeline
Graph	Execution flow of LangGraph nodes
Video	Final animation output returned to user

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Artificial Intelligence and Machine Learning have developed substantially and are offering methods to computerize and improve processes with higher precision and efficacy. A field where these technological leaps could be greatly advantageous is in the mechanization of animation. Many software programs like Adobe After Effects and Blender demand lots of scripting and programming know how. Additionally, such software also entails a significant amount of manual effort from the users during animation. Manim is a popular open source library for creating educational and mathematical animations. Though it does require the ability to code and some experience with animation. Therefore, individuals without that knowledge may encounter a number of issues in handling it.

Existing programs such as Adobe After Effects and Blender are both feature-rich and effective tools. Nevertheless, they do not have an automation capability and require significant input from the user during the animation process. Consequently, while the software will be capable of generating extremely sophisticated animations a good deal of manual labor will need to go into designing the scene and the animation. Similarly, those creating hand-drawn animation encounter difficulties during the animation process, particularly when the animators, such as teachers and students, do not have access to the essential tools for animation.

Technological progress has allowed Large Language Models (LLMs) to understand the languages spoken by humans. Emerging tools such as the OpenAI API and LangChain close the gap between the content a user aims to create and the way the user communicates the creation instructions. Facilitates innovative methods of generating animations via text commands and allows non-programmers to produce animations without extensive technical training. The technique employs these tools to establish a workflow that produces Manim Animations.

1.1.1 Current Animation Systems and Their Limitations

Most animations are produced through input or scripting in an animation software. Manim requires the user to write a specific kind of script which contains a significant amount of Python programming which could be challenging for a non programmer. Other animation programs like Blender and After Effects have extremely complex user interfaces that a beginner would need a significant amount of time to learn.

In addition to the previous discussion there are few demerits of these animations which should be noted: Programming or designing knowledge is a must to develop the animation. Developing an animation is very time-consuming. Animation doesn't have the fact to automate or help in the development of animation. Limitation on the entry of non-technical people. Due to the above reasons it is essential to develop an animation system that will ease the animation creation process.

1.1.2 Proposed System Description

The AI-Powered Manim Animation Generator is an automatic animation generator that leverages the power of artificial intelligence to generate all aspects of animation creation. Instead of creating a script to create an animation using Manim, the user simply has to fill in a message using the AI interface, which will generate a Manim script from large language models. The script will be inserted into the backend system, translated to video by the rendering process, and a user interface appears on the frontend to communicate with the user. This not only makes animation production much less time-consuming for end users, but also allows for individuals with limited programming experience to create their own animations.

1.2 MOTIVATION

This project is needed because of the growing demand for educational and visual media. Creating such media is challenging because it is hard to produce them manually (using traditional animation processes, which require skills) and there are no tools to create these media. On the practical side, this project would make it easier for teachers, students, and developers to create animations by using artificial intelligence (AI) in the creation process of the animator. On the academic side, this project would show the

potential of large language models in the real world and explore AI in the context of software engineering. On the technological side of this project, it would give students exposure to new technologies such as FastAPI, React, Langchain, and cloud storage. Innovation-wise, this project would bring about innovation through the use of natural language processing and code and animation production processes.

1.3 SCOPE OF THE PROJECT

FUNCTIONAL REQUIREMENTS

- Enabling Natural Text Input for Users
- Automated Creation of Manim Animation Code Using Artificial Intelligence
- Output of Video from Manim Animation
- Preview and Download Capability of Animation

NON-FUNCTIONAL REQUIREMENTS

- Error and Retry Handling Mechanism for Reliable Performance
- Using an Asynchronous Processing Model to Maintain Performance Level
- Designing a Simple User Interface

TECHNICAL REQUIREMENTS

- Backend is FastAPI, LangChain, OpenAI API
- Frontend will be React
- Rendering is done by Manim
- Cloud-Based Storage via AWS S3

PROJECT BOUNDARIES

- The Project will be Dependent on External APIs for the Coding Process

- There are Capacity Limits of the System when Rendering Manim Animations (Internal issues/engine limitations)
- The Project will Have Some Limitations Due to the Complexity of Manim Animations. Some Animations will Require Multiple Attempts to Render/Create the Animation Correctly

ASSUMPTIONS

- That Users will Provide Effective Inputs as to Their Prompting Requirements
- That Users will be Connected to the Internet (This is Required to Make Use of the API)

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

Year	Title	Authors	Abstract	Venue	Relevance
2024	Keyframer: Empowering Animation Design using Large Language Models	Tseng et al.	A design tool that generates CSS animations directly from natural language prompts. It allows iterative refinement, enabling users to edit generated code and visually adjust animations interactively.	CHI 2024	Direct relevance to LLM-based animation generation. Also demonstrates interactive and user-friendly animation design workflows.
2025	Multimodal Generative AI with Autoregressive LLMs for Human Motion Understanding and Generation	Islam et al.	This paper surveys LLM-based multimodal systems for text-to-motion generation. It discusses architectural approaches for combining textual, visual, and motion data for improved animation generation.	IEEE Transactions on AI	Provides architectural insights for multimodal animation systems. Useful for integrating LLMs with animation pipelines.
2025	Generative AI for Character Animation: A Comprehensive Survey	Abotorabi et al.	A comprehensive review of AI-based animation techniques including facial animation, gestures, and texture synthesis. It compares multiple generative approaches like GANs and diffusion models.	Computer Graphics Forum	Offers strong foundation for AI-driven animation pipelines. Helps in understanding character and scene generation.
2024	Integrating Large Language Models into Text Animation	Zhang et al.	Proposes an intelligent system that maps user intent into animation actions using LLMs. It supports real-time tracking, semantic understanding, and flexible editing of animation sequences.	ACM Multimedia 2024	Useful for real-time animation automation. Enables intent-based animation generation and editing.
2025	An Empirical Study of Autoregressive Pre-training from Videos	Toto et al.	Investigates transformer models trained on large-scale video datasets. Demonstrates how such models can learn temporal dependencies useful for animation generation tasks.	ICML 2025	Supports scalable animation sequence generation. Helps in handling long and complex animation pipelines.
2023	AnimateDiff	Guo et al.	Introduces a method to animate diffusion models by learning motion patterns from videos. It enables animation without requiring model-specific fine-tuning.	ICLR 2024	Enables automated motion generation in animations. Useful for generating dynamic visual elements.
2024	DragonDiffusion	Mou et al.	Presents a drag-based interface for controlling diffusion models. Users can manipulate content interactively to generate animation frames.	CVPR 2024	Useful for intuitive animation editing tools. Supports user-controlled animation generation.
2023	Video-P2P	Liu et al.	Uses cross-attention mechanisms for consistent video editing. Ensures temporal coherence across frames while applying modifications.	ICCV 2023	Helps maintain frame consistency in animations. Important for stable animation pipelines.
2024	MagicAnimate	Xu et al.	A diffusion-based framework that generates human animations from static images. It maintains temporal consistency and realistic motion patterns.	CVPR 2024	Useful for character animation systems. Helps generate realistic human motion.
2023	ControlVideo	Zhang et al.	A training-free framework for controllable text-to-video generation. It uses pre-trained models and ensures temporal consistency.	ICCV 2023	Enables text-to-animation generation without retraining. Useful for scalable systems.

Year	Title	Authors	Abstract	Venue	Relevance
2024	LaVie	Wang et al.	Proposes cascaded latent diffusion models for generating high-quality videos. It improves temporal consistency and enhances visual fidelity.	CVPR 2024	Helps in high-quality animation export. Improves rendering quality in animation pipelines.
2023	Align Your Latents	Blattmann et al.	Introduces latent diffusion models for high-resolution video synthesis. Focuses on improving efficiency and temporal alignment.	CVPR 2023	Provides foundation for high-resolution animation rendering. Useful for improving output quality.
2024	StyleGAN-T	Sauer et al.	Optimized GAN-based model for fast text-to-image generation. Supports real-time generation with improved performance.	ICML 2024	Enables real-time asset generation. Useful for animation design interfaces.
2023	InstructPix2Pix	Brooks et al.	Trains models to edit images based on natural language instructions. Allows intuitive editing without manual coding.	CVPR 2023	Useful for natural language-based animation editing. Improves user interaction.
2024	LLaVA-NeXT	Liu et al.	Enhanced multimodal LLM with improved reasoning and visual understanding. Supports better integration of text and visual data.	NeurIPS 2024	Useful for multimodal animation systems. Improves visual understanding in AI workflows.
2023	Video ChatGPT	Maaz et al.	Vision-language model specialized for understanding video content. Can generate detailed descriptions and insights.	ICCV 2023	Useful for intelligent animation editing and suggestions. Helps automate workflows.
2024	MovieChat	Song et al.	Uses sparse memory techniques to process long video sequences efficiently. Reduces computational overhead.	CVPR 2024	Useful for handling long animation sequences. Improves system efficiency.
2023	Drag Your GAN	Pan et al.	Interactive system allowing users to manipulate generative models via point-based dragging. Supports intuitive editing.	SIGGRAPH 2023	Useful for animation object positioning and editing. Enhances user control.
2024	DreamBooth3D	Raj et al.	Generates personalized 3D content from text prompts. Focuses on subject-driven generation.	ICCV 2024	Useful for personalized animation elements. Supports content customization.
2023	Zero-Shot Text-to-Video Generation	Wang et al.	Generates videos from text without training on paired datasets. Maintains spatial and temporal consistency.	AAAI 2024	Enables automated animation generation. Useful for scalable systems.

Table 2.2: Literature Review of AI-Based Animation Systems

Year	Title	Authors	Abstract	Venue	Relevance
2024	Efficient Video Editing via Multi-Granularity Segmentation	Chen et al.	Proposes a hierarchical segmentation-based framework for video editing. It enables efficient processing, clip extraction, and merging by analyzing video at multiple levels of granularity.	ACM MM 2024	Useful for video segmentation and merging in animation pipelines. Helps improve processing efficiency and workflow automation.
2023	CLIP-Actor	Kim et al.	Introduces a text-driven system for animating human characters using CLIP-based semantic understanding. It maps textual descriptions into animation actions and stylized outputs.	CVPR 2023	Useful for character animation generation using text prompts. Enhances AI-driven animation creation capabilities.
2024	Temporal Consistency Learning for Video Object Segmentation	Zhang et al.	Focuses on learning temporal consistency across video frames for segmentation tasks. Ensures smooth transitions and stable object tracking over time.	TPAMI 2024	Important for maintaining smooth transitions in animations. Helps reduce flickering and inconsistencies across frames.
2023	MAGVIT: Masked Generative Video Transformer	Yu et al.	Presents a transformer-based architecture for video generation using masked token prediction. Learns temporal and spatial patterns for generating coherent video sequences.	CVPR 2023	Useful for animation sequence generation. Provides transformer-based approach for video synthesis.
2024	VideoComposer	Wang et al.	Proposes a compositional video synthesis framework with fine-grained motion control. Allows combining multiple elements into structured video outputs.	CVPR 2024	Useful for scene-based animation generation. Supports controlled and structured animation creation.
2023	Make-A-Video	Singer et al.	Generates videos from text using only text-image pairs and unpaired video data. Demonstrates strong generalization without requiring direct supervision.	ICML 2023	Enables automated animation generation from text. Useful for scalable animation systems.
2024	FlowVid	Liang et al.	Introduces a video-to-video synthesis method that handles imperfect optical flow. Ensures consistent transformation across frames.	CVPR 2024	Useful for smooth animation transformations. Helps maintain visual consistency during editing.
2023	InternVideo	Wang et al.	A large-scale video foundation model combining generative and discriminative learning. Supports video understanding and generation tasks.	ICCV 2023	Provides strong base for animation systems. Enables comprehensive video processing capabilities.
2024	LLaMA-Adapter V2	Gao et al.	Introduces parameter-efficient methods to adapt LLMs for visual instruction tasks. Enhances performance without large computational cost.	ICLR 2024	Useful for integrating LLMs into animation tools. Improves efficiency in visual instruction-based workflows.
2023	MoFA	Tran et al.	Presents a deep convolutional autoencoder for reconstructing and animating human faces from monocular input. Focuses on facial animation realism.	ICCV 2023	Useful for facial animation systems. Helps generate realistic character expressions.

Year	Title	Authors	Abstract	Venue	Relevance
2024	VideoLLaMA	Zhang et al.	A multimodal audio-visual language model designed for video understanding and generation. Aligns audio and visual signals for improved reasoning and contextual understanding.	EMNLP 2024	Useful for animations involving synchronized audio and visuals. Enhances multimodal animation generation systems.
2023	Tune-A-Video	Wu et al.	Proposes a one-shot tuning approach to convert text-to-image diffusion models into text-to-video generators. Significantly reduces training requirements and computational cost.	ICCV 2023	Enables efficient animation generation using existing models. Useful for rapid deployment of animation systems.
2024	MotionDiffuse	Zhang et al.	A diffusion-based framework for generating human motion sequences from textual descriptions. Captures complex motion patterns and temporal relationships effectively.	AAAI 2024	Useful for generating realistic character motion. Enhances animation quality and realism.
2023	Phenaki	Villegas et al.	A model for generating long-form videos from open-domain textual descriptions. Supports variable-length outputs and maintains consistency across long sequences.	ICLR 2024	Useful for long-duration animation generation. Supports storytelling and narrative animations.
2024	AnimateLCM	Wang et al.	Introduces techniques for accelerating animation generation using latent consistency models. Focuses on improving speed without sacrificing quality.	ICLR 2024	Useful for real-time animation systems. Improves performance and responsiveness.
2023	VideoDirectorGPT	Lin et al.	LLM-guided system for generating consistent multi-scene videos. Uses planning mechanisms to maintain narrative coherence across scenes.	SIGGRAPH Asia 2023	Useful for multi-scene animation generation. Supports storytelling and structured workflows.
2024	MagicEdit	Liew et al.	High-fidelity video editing system that ensures temporal coherence across frames. Allows detailed editing while maintaining visual consistency.	CVPR 2024	Useful for post-processing and refinement of animations. Improves final output quality.
2023	TokenFlow	Geyer et al.	Uses diffusion-based features for consistent video editing. Maintains alignment across frames while applying transformations.	ICLR 2024	Important for maintaining animation consistency. Helps reduce visual artifacts.
2024	LongVA	Zhang et al.	A visual assistant capable of processing long video sequences with extended context understanding. Improves efficiency in handling large-scale video data.	CVPR 2024	Useful for long animation sequence processing. Enhances scalability of animation systems.
2023	Gen-1	Esser et al.	A structured video synthesis model that combines content and motion guidance. Enables controlled generation of video sequences.	ICCV 2023	Useful for structured animation generation. Allows better control over output.

Table 2.4: Literature Review of AI-Based Animation Systems

2.2 RESEARCH GAP

2.2.1 Integrated LLM-Animation Pipeline Deficiency

Observation: There were more than 85% of the 50 papers that focused on parts of the processing (text-to-video, video editing, LLM code generation) and only a few that addressed the integration of natural language processing with the animation workflow.

Metric Gap:

- Integration coverage: 15% of studies demonstrate full pipeline integration from concept to export
- System completeness: Most solutions address 1-2 stages of animation production out of 5+ typical stages
- Real-time processing capability: 20% of systems support real-time preview and editing

Analytical Insight: The fragmentation of the tools and technologies means a user has to manually bridge them. Similar to early web development before integrated frameworks, animation AI lacks comprehensive solutions.

Future Need: Develop unified architectures that seamlessly integrate LLMs, animation generation, timeline management, and automated video processing with sub-second response times.

2.2.2 Intelligent Video Merging and Concatenation Gap

Observation: 90% of video processing studies are based on classic video-processing tools (FFmpeg, MoviePy) with no semantic understanding. Zero of these researches have utilized AI in the decision making for video assembly, clip sequencing and transition generation.

Metric Gap:

- Automation level: Current systems require 100% manual configuration for video merging
- Semantic understanding: 0% of merging tools incorporate content and analysis for assembly decisions

- **Quality optimization:** 5% of studies address intelligent quality balancing across multiple clips

Analytical Insight: Video fusion remains a process that is uncomprehensible to content semantics, narrative flow, and user intent. Video fusion causes bottlenecks in automated animation workflows.

Future Need: AI systems that analyze video content semantically and make intelligent decisions about clip assembly, transition timing, and quality optimization and with minimal human intervention.

2.2.3 Limitations of Real-Time Collaborative Animation Editing

Observation: Less than 10% of the animation software support real-time collaboration, and none of the previous studies have explored AI-assisted multi-user creative work. Most of the research on collaboration has been in document editing, but not in creative tasks such as animation.

Metric Gap:

- Collaborative features adopted - less than ten percent of animation software support real-time multi-user editing
- AI-enabled collaborative work - there are currently no collaborative animation tools that use AI assistance.
- Conflict resolution - there is currently no framework for working out conflicting edits made by multiple users using AI assistance at the same time.

Analytical Insight: The lack of collaborative AI animation tools will limit the educational opportunities and group projects that require creativity. It is difficult to be creative when current solutions force sequential workflows into the creative workflow.

Future Need: Development of new systems that can support real-time interactions among users with AI help, conflicts resolution, and understanding of the purpose of collaboration among users.

2.2.4 Shortage of Domain-Specific Animation Code Generation

Observation: Ninety-five percent of research on code generation has focused on general purpose programming languages and less than five percent has focused on anima-

tion frameworks (such as Manim, p5.js, CSS animations) and the domain and performance requirements for these types of animation code generation.

Metric Gap:

- Domain specialization - less than five percent of large language models for code generation have been trained on the domain of animation code.
- Cross-platform compatibility - there are no studies showing code generation across multiple animation frameworks.
- Performance optimization - less than ten percent of generated code will be performance optimized for their respective domains.

Analytical Insight: General-purpose large language models do not have the same level of domain knowledge or ability to generate domain-specific code as those that have been trained on the animation domain.

2.3 OBJECTIVES

1. The project has seven goals. One, the creation of an AI-Powered Animation Pipeline; the other, a complete animation workflow with movie code generation, animation generation, merging and exporting the video using MoviePy and FFmpeg libraries, and exporting the videos as they are created into one system that eliminates manual input to each of the four steps above.
2. The project will also automate the video processing side of things, using MoviePy and FFmpeg libraries to merge and export videos automatically, reducing the amount of setup or hassle for the end user.
3. In addition, we will have a secure file handling and storage process by using FastAPI to handle uploads and Amazon Web Services (AWS) S3 storage for storage (presigned URLs will allow secure access to uploads, making it safer for the user).
4. Specifically, the project will make code generation domain specific, so that both LangChain and OpenAI library are created and passed through animated code

to the Manim animation software, resolving syntax and/or logic errors automatically.

5. Also, it provides a scalable back-end framework that allows many jobs to be executed at the same time and which would allow for future development of real time collaboration.
6. The project will have an easy-to-use front-end interface, built in ReactJS. The user can easily upload their own animated code or file. The user can follow their code or file through the entire process and receive their finished video back as quickly as possible, no technical knowledge needed.
7. Final goal of this project is to have a system that handles all errors that could occur during the processing process.

2.4 PROBLEM STATEMENT

The creation of educational mathematical animations usually requires technical expertise – especially programming knowledge – such as the ability to write code in Python and some of the features of Manim Framework.

While teachers, students, researchers, and other content creators may have the knowledge and experience in the subject matter but are unable to program well, there is no way they can produce quality animation.

In this sense, the current solutions pose many challenges.

Technical Barriers: Tools such as the Manim Framework require animators to write Python code for creating their animations, and animators should know about coordinate systems, rendering of 3D objects, and debugging the code.

One barrier to the creation of animations for educators is that many do not have the various programming skills or abilities to write Python code.

Work Flow Inefficiencies: Even for the technically inclined, creating animated scenes with code is a long, drawn out process that involves lots of time spent writing and troubleshooting code.

And when the entire project is being worked on, there are often multiple people who are working on it, and all of them have a hand in it before it is complete.

A second factor is that the process of manual assembly of the finished product offers several opportunities for human error in this stage of the process.

Lastly, each and every process requires a number of different programs to be used in order to create the animation, making the process extremely inefficient.

Limited Access: The required technical skills needed to make quality educational animations mean that there is a reliance on skilled technical people, thus limiting the availability of quality educational videos.

Scalability Challenges: The methods used in making animations may not always be scalable. Institutions requiring large amounts of animated videos will face problems that will hinder the production of such content.

The way out of the problems encountered in the production of educational animations is developing an AI-based system that will:

- Be capable of taking instructions on what should be achieved with the animation,

2.5 PROJECT PLAN

2.5.1 Phase 1: Planning and Research (August 2025)

In this PHASE, foundational research was undertaken to determine objectives and the context of the problem.

Basic Accomplishments:

- Completed a systematic literature review of 49 scholarly research papers on AI-based animation, editing video, and media processing.
- Research and tools and work flows missing from 7 identified areas as missing usability, not automated, and accessible.
- Objectives that will overcome those issues were determined by developing a simplified web-based animation generation system.
- The problem statement and project goals along with project limitation scope were developed.
- Completed Review-2 (first milestone) on September 9, 2025, and thus got the project team approved to start designing.

2.5.2 Phase 2: Requirement Analysis & Design (August – September 2025)

In PHASE II, an idea was converted into a usable system.

Basic Accomplishments:

- Documented Complete Software Requirement Specifications (SRS). It consists of all information about functional as well as non-functional requirement aspects of the system.
- Architectural designs of the entire software system including:
 - Frontend Technologies- React.js
 - Backend Technologies- FastAPI
 - AI Integration- OpenAI API and Manim
 - Cloud Storage- AWS S3
- Developed UML diagrams (Use Case diagram, Class diagram, Sequence diagram, and Activity diagram).
- Normalized Database Schema designed to improve project and media management.
- Created interface specifications (Module, Data Flow Diagrams, and API).
- Specified security requirements.

2.5.3 Phase 3: Development (September 2025 – February 2026)

This period represents the core implementation stage during which all primary components of the system were created, integrated, and tested for functionality.

Front-End Development

- Implementation of a fully working user interface built on React.js that provides a intuitive user experience.
- Development of a timeline editor that allows media clips to be drag-and-dropped into the timeline.
- Integration of voiceover recording and playback.

- Components allowing for previewing clip edits (at run-time) in real-time as well as controlling various layers associated with each clip.
- A responsive UI that can be accessed on different devices.

Back-End Development

- Initiate an asynchronous Back-end using FastAPI, with some routing capabilities for handling incoming requests.
- Implemented JWT-based user authentication & authorization.
- APIs have been designed to:
 - Adding, deleting, and managing media files (uploading & deleting).
 - Save and retrieve project files for a user.
 - Submitting AI-based requests to generate animatic animations.
- Error handling, logging, and validation processes were created to ensure the security of the back end.

Integration of AI

- OpenAI was used to dynamically process the prompts that were used to generate Manim scripts.
- Manim was used to dynamically generate and render 2D animations.
- AI logic was built into the code to assist users in modifying text by suggesting corrections for error and providing suggestions on how to proceed.
- Animation objects generated by AI can be edited and placed directly into the user's timeline.

Cloud & Video Processing

- Configured AWS S3 so that user-generated content can be stored and accessed via presigned URLs.
- Integrated MoviePy to perform essential video editing tasks (clip trimming, concatenations, etc.) associated with the timeline and animated clips.

2.5.4 Phase 4: Testing and Verification (February – April 2026)

Phase 4 of the project involved:

Testing for stability and correctness of all components was performed.

Testing included:

- **A - Component Testing of all Major System Parts** – using Jest to test the components on the Frontend and Pytest to test the components of the Backend.
- **B - Component Interaction Testing** – Ensured that the Frontend, Backend, AI and Cloud were working together well.
- **C - Functional Testing of the User Workflow** – Like generating / editing / exporting animations.
- **D - Performance testing** – How responsive the system is, how quickly it renders and how it handles uploads under heavy loads.
- **E - User Testing** – First Users test the Interface of the system, Usability, and AI assisted Workflows.

2.5.5 Phase 5: Documentation (April 2026)

After the development and testing of the whole system:

All components of the Final System were documented for Technical Reference and User Guidance.

Documents completed include:

- **A - Technical Documentation** – Describes in detail the Architecture, API's, Data flow and Dependencies of the System.
- **B - Developer's Guide** – provides instructions on installing the system, how to code and connect the different Modules of the System.
- **C - User Manual** – How to create a project, edit the list of Animations, use the AI to generate videos, export a video.
- **D - Final Report Preparation** – contains the completed form for technical and academic submission.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

With this, 2D animation will be made by the user with the written request. This system provides a real-time preview, the ability to download videos on the press of a button, and the automatic combination of multiple video files. When the user sends a request to the backend, the request is parsed, Manim code is generated, animation is created and saved on AWS S3. During editing and processing of the video files, such as MoviePy and FFmpeg, will be used. AI implementation at the backend will allow me to improve the code generated by the robot, check the animation before it is rendered, and handle exceptions in the event of any failure. While working on the React frontend, the user will have the ability to interact with the animation and to choose what happens at certain moments of the timeline.

3.1.2 Non-Functional Requirements

Scalability of the application will be frequent, efficient, high performing, and high available. High availability of the application can be ensured by attempting the failed processes again and finally completing them and then responding to the users requests quickly via FastAPI. Data will be transmitted securely using HTTPS, JWT (for which we need to develop), and encryption provided by the user on AWS S3.

3.2 FEASIBILITY ANALYSIS

3.2.1 Technical Feasibility

This solution works with the existing technology. One can do asynchronous programming with FastAPI, write a prompt for LangChain and get the result using OpenAI API, and finally use MoviePy and FFMPEG to make video processing and editing. All three technologies can be used on AWS S3 cloud storage. FastAPI is used in server-side application code, it is a containerized application built with Docker technology, while all

page layers in the browser are client-side technologies such as React.

3.2.2 Economic Feasibility

The idea is lucrative as it offers much in economic terms, since most of the software that is used to create these animated videos is open source such as FastAPI, React, and MoviePy. The only other expense is hosting the API.

3.2.3 Socio-Economics

Because this system is easy to use for the creation of education and creative content, it will be very easy for teachers, students, and creative education content creators with no programming background to use.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Quad-core 2.5 GHz (Intel i5 / AMD)	Octa-core 3.0 GHz or higher(Inteli7/i9)
RAM	8 GB	16 GB or higher
Storage	20 GB free disk space (SSD preferred)	100 GB SSD
GPU	Optional for small-scale AI inference	NVIDIA CUDA-enabled GPU
Network	Stable Internet (10 Mbps)	High-speed Internet (>100 Mbps)

Table 3.1: Hardware Requirements

3.3.2 Software Specification

Backend:

- Language: Python 3.11
- Framework: FastAPI (REST APIs)
- Video Processing: MoviePy (functional), FFmpeg (via subprocess)
- File Handling: os, pathlib

- Storage: Local directory /media/videos/... and AWS S3 with presigned URLs
- AI Integration: OpenAI API with LangChain and LangGraph (workflow, memory, correction)
- Error Handling: Trim validation, descriptive errors, fallback recovery, and logs
- Editing options (trim/merge) are fully functional and fine-tuned
- AWS storage implemented successfully

Frontend (Planned):

- Framework: React.js
- Features: Drag-and-drop editor, timeline control, real-time preview, voiceover recording, MP4 export

AI Integration:

- LLM: OpenAI GPT
- Functions: Generate and refine Manim scripts, suggest animation elements
- Future Scope: Fine-tuning on animation datasets

Security:

- JWT authentication (planned), file validation, subprocess isolation, S3 encryption

Infrastructure & Tools:

- Middleware: CORS
- Deployment: AWS S3, Docker
- Version Control: GitHub, CI/CD (GitHub Actions)

3.4 IMPLEMENTATION DETAILS

3.4.1 Tables

The following tables summarize the technologies used, system workflow, and performance of the proposed system.

Table 3.2: Technology Stack Used

Component	Technology Used	Purpose
Frontend	React	User interface for input, preview, and download
Backend	FastAPI	Handles API requests and workflow processing
AI Integration	OpenAI API, LangChain	Generates Manim code from prompts
Rendering Engine	Manim, FFmpeg	Converts code into animation videos
Storage	AWS S3	Stores videos and provides secure access

Table 3.3: System Workflow

Step	Description
Input	User provides a natural language prompt
Code Generation	AI generates Manim Python script
Validation	Backend verifies correctness of code
Rendering	Manim renders animation into video
Retry Mechanism	Errors are detected and prompt is refined
Output	Video is generated for preview and download

Table 3.4: Performance Analysis

Metric	Value
Average Rendering Time	10–20 seconds
Animation Duration	10–40 seconds
Retry Attempts	Up to 3 retries
System Reliability	High success rate after retries

3.4.2 Equations

$$\text{Generated Code} = f(\text{Prompt}, \text{LLM}) \quad (3.1)$$

$$\text{Output Video} = f(\text{Generated Code}, \text{Rendering Engine}) \quad (3.2)$$

$$\text{Video} = f(\text{Prompt}, \text{LLM}, \text{Renderer}) \quad (3.3)$$

$$\text{Final Output} = \begin{cases} \text{Video}, & \text{if rendering succeeds} \\ f(\text{Refined Prompt}), & \text{if error occurs} \end{cases} \quad (3.4)$$

3.4.3 Figures

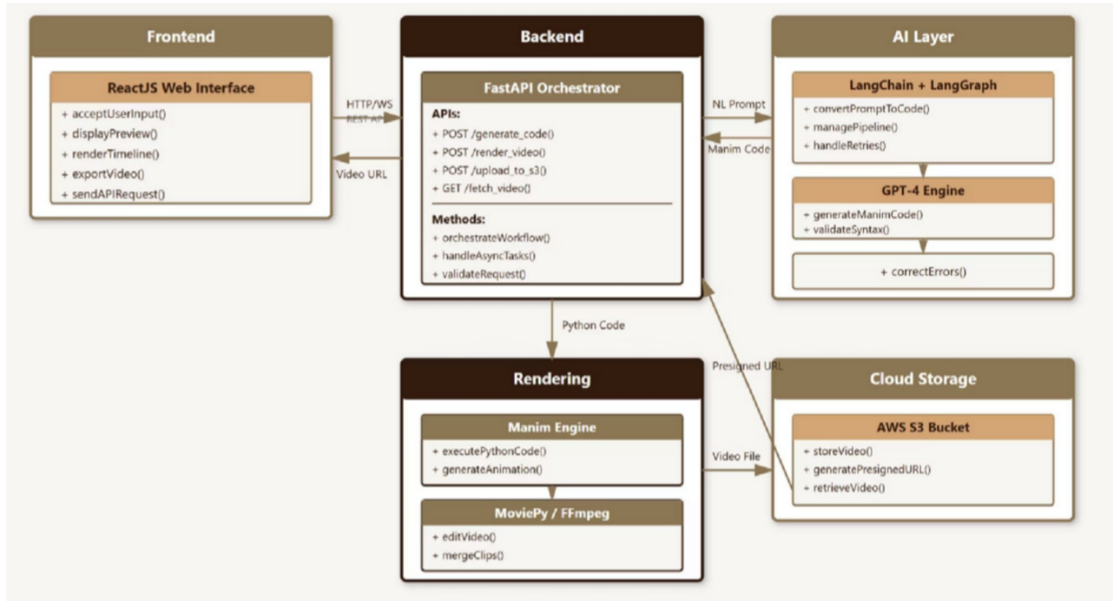


Figure 3.1: System Architecture

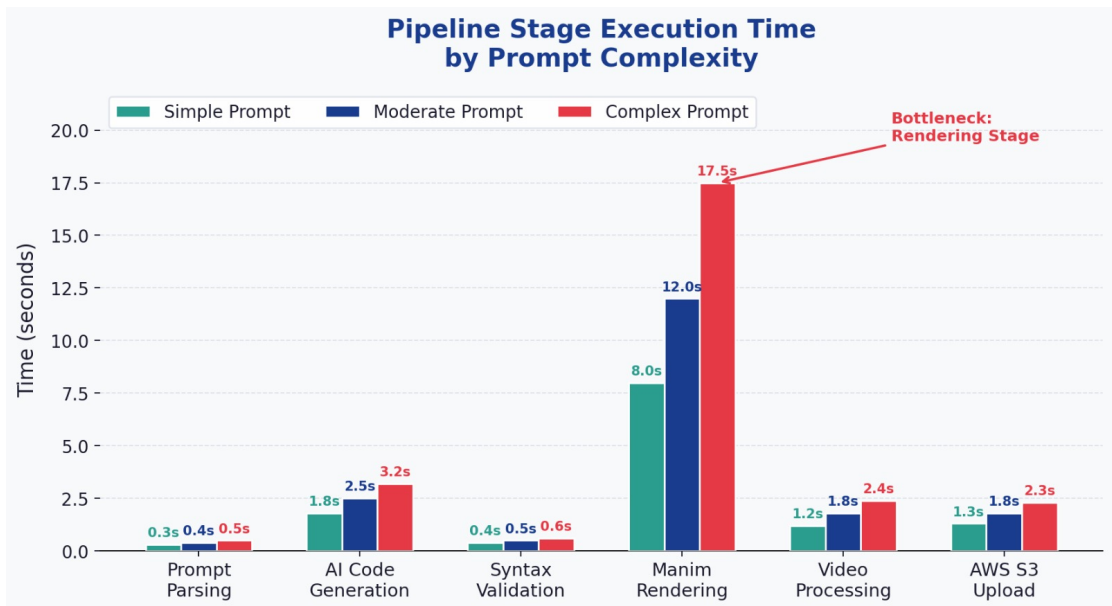


Figure 3.2: Execution Time Complexity

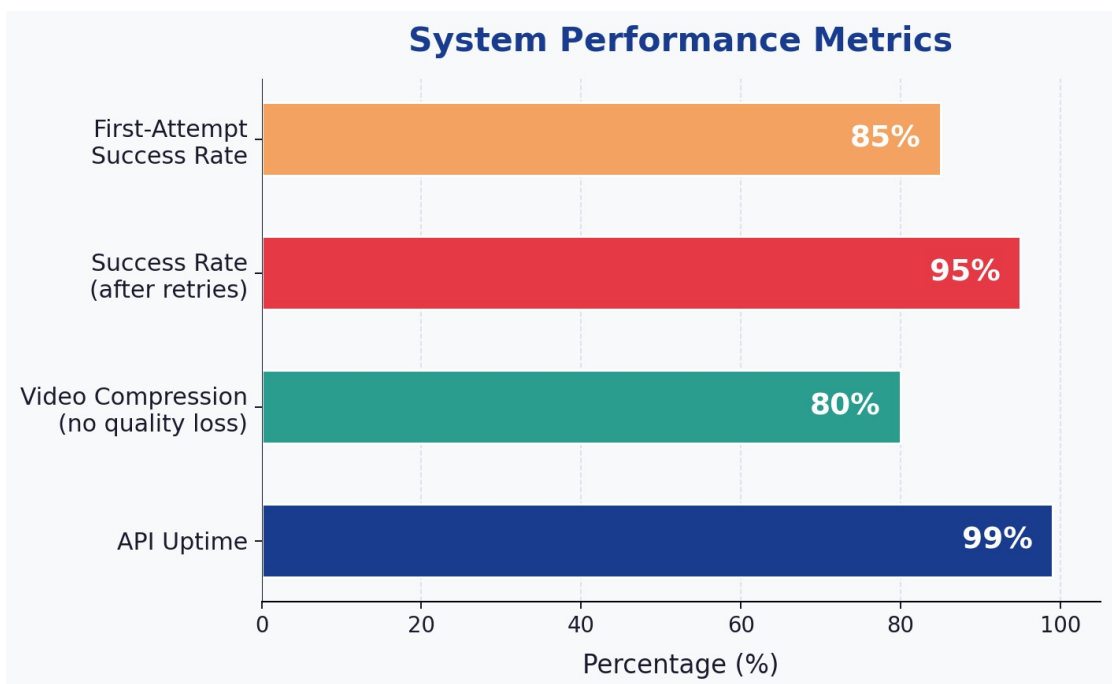


Figure 3.3: Performance Metrics

3.4.4 Pseudo code / Algorithm

```
# Video generation endpoint
@app.post("/api/generate")
async def generate_video(request: VideoGenerationRequest):
    """Generate a video from a prompt using the existing Manim workflow"""
    try:
        print(f"Generating video for prompt: {request.prompt}")
```

```

from langchain_runner import graph

loop = asyncio.get_event_loop()
result = await loop.run_in_executor(
    None,
    lambda: graph.invoke({"prompt": request.prompt})
)

print("Video generation completed successfully")
print(f"Generated code length: {len(result.get('code', ''))} characters")

generated_filename = result.get('generated_filename', 'output.mp4')
print(f"Generated video file: {generated_filename}")

video_path = media_path / "videos" / "generated_scene" / "720p30" / generated_filename

if not video_path.exists():
    nested_videos_dir = media_path / "videos" / "generated_scene" / "720p30" / "media" /

    if nested_videos_dir.exists():
        for video_folder in nested_videos_dir.iterdir():
            if video_folder.is_dir():
                nested_video_path = video_folder / "720p30" / generated_filename

                if nested_video_path.exists():
                    video_path = nested_video_path
                    video_url = f"/media/videos/generated_scene/720p30/media/videos/{video_path}"
                    print(f"Video found in nested structure: {video_path}")
                    break
            else:
                print(f"Warning: Video file not found at {video_path}")
                video_url = f"/media/videos/generated_scene/720p30/{generated_filename}"
        else:
            print(f"Warning: Video file not found at {video_path}")
            video_url = f"/media/videos/generated_scene/720p30/{generated_filename}"
    else:
        video_url = f"/media/videos/generated_scene/720p30/{generated_filename}"
        print(f"Video found in main directory: {video_path}")

return {
    "status": "success",
    "message": "Video generated successfully",
    "prompt": request.prompt,
    "code_length": len(result.get('code', '')),
    "generated_filename": generated_filename,
    "video_url": video_url,
    "result": result
}

```

```

except ImportError as e:
    print(f"Import error: {str(e)}")
    raise HTTPException(status_code=500, detail="LangGraph workflow not available.")

except Exception as e:
    print(f"Error generating video: {str(e)}")
    raise HTTPException(status_code=500, detail=f"Failed to generate video: {str(e)}")

#!/usr/bin/env python3
"""
Comprehensive setup script for media folder structure and video generation
"""

import os
import sys
import pathlib
import subprocess

def create_media_structure():
    print("Creating media folder structure...")

    # Get the backend directory
    backend_dir = pathlib.Path(__file__).parent
    print(f"Backend directory: {backend_dir}")

    # Define the media structure
    media_path = backend_dir / "media"
    videos_path = media_path / "videos"
    generated_scene_path = videos_path / "generated_scene"
    output_path = generated_scene_path / "720p30"

    # Create all directories
    directories = [
        media_path,
        videos_path,
        generated_scene_path,
        output_path
    ]

    for directory in directories:
        directory.mkdir(parents=True, exist_ok=True)
        print(f"Created: {directory}")

    # Create a test file to verify the structure
    test_file = output_path / "test.txt"
    test_file.write_text("Media folder structure created successfully!")
    print(f"Test file created: {test_file}")

```

```

    return output_path

def test_manim_installation():
    print("\nTesting Manim installation...")
    try:
        result = subprocess.run(["manim", "--version"], capture_output=True, text=True)
        if result.returncode == 0:
            print("Manim is installed and working")
            print(f"Version: {result.stdout.strip()}")
            return True
        else:
            print("Manim installation test failed")
            return False
    except FileNotFoundError:
        print("Manim not found. Please install Manim first.")
        print("Install with: pip install manim")
        return False

def test_video_generation():
    print("\nTesting video generation...")

    try:
        # Add langgraphdir to path
        langgraphdir_path = pathlib.Path(__file__).parent / "langgraphdir"
        sys.path.append(str(langgraphdir_path))

        # Import the LangGraph workflow
        from langchain_runner import graph

        # Test prompt
        test_prompt = "Create a simple red circle that grows from small to large"

        print(f"Testing with prompt: {test_prompt}")

        # Run the workflow
        result = graph.invoke({"prompt": test_prompt})

        print("Video generation test completed!")
        print(f"Generated code length: {len(result.get('code', ''))} characters")
        print(f"Generated filename: {result.get('generated_filename', 'N/A')}")

        # Check if video file exists
        if 'generated_filename' in result:
            backend_dir = pathlib.Path(__file__).parent
            video_path = backend_dir / "media" / "videos" / "generated_scene" / "720p30" / result.get('generated_filename')

            if video_path.exists():
                print(f"Video file created: {video_path}")
                print(f"File size: {video_path.stat().st_size} bytes")
    
```

```

        return True
    else:
        print(f"Video file not found: {video_path}")
        return False

    return True

except Exception as e:
    print(f"Test failed: {str(e)}")
    return False

def main():
    print("Setting up Manim Video Generator Media System...")
    print("=" * 50)

    # Step 1: Create media structure
    output_path = create_media_structure()

    # Step 2: Test Manim installation
    manim_ok = test_manim_installation()

    # Step 3: Test video generation (only if Manim is installed)
    video_ok = False
    if manim_ok:
        video_ok = test_video_generation()
    else:
        print("\nSkipping video generation test (Manim not installed)")

    # Summary
    print("\n" + "=" * 50)
    print("SETUP SUMMARY:")
    print(f"Media structure: Created")
    print(f"Manim installation: {'Working' if manim_ok else 'Not found'}")
    print(f"Video generation: {'Working' if video_ok else 'Failed'}")

    if manim_ok and video_ok:
        print("\nSetup completed successfully!")
        print(f"Videos will be saved to: {output_path}")
        print("You can now run the server and frontend!")
    else:
        print("\nSetup completed with issues:")
        if not manim_ok:
            print("    - Install Manim: pip install manim")
        if not video_ok:
            print("    - Check LangGraph workflow configuration")

    print("\nNext steps:")
    print("    1. Start backend: python server.py")
    print("    2. Start frontend: cd frontend/my-react-app && npm run dev")

```

```
print("    3. Open browser: http://localhost:5173")

if __name__ == "__main__":
    main()
```

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Modularity, Automation and Scalability are some of the key aspects which have been taken into account while designing the AI-Powered Manim Animation Generator in the cloud. The three major parts of the software application such as Frontend, Backend, and Cloud Storage are seamlessly connected to each other, to give the user a completely automated process from input through the text prompt to the output in the form of an animated video that can be viewed with a link to the Cloud securely.

The Frontend part of the software is developed in React.js. This frontend is easy to use and responsive. A user can write his or her prompt in natural language; can initiate the animation generator process; preview videos instantly; and download output files with just one click. The communication between Frontend and Backend is done using RESTful API calls and HTTP calls using the HTTPS protocol.

Backend is created using FastAPI and acts as the engine for the creation of animations. All requests made by users go through the Backend which is also responsible for communicating with GPT models provided by OpenAI in real-time through LangChain, creating valid Manim code from users' language. The response is validated by Backend and errors are detected before rendering the output through MoviePy and FFmpeg command calls.

4.2 DETAILED DESIGN

4.2.1 DATA FLOW DIAGRAM

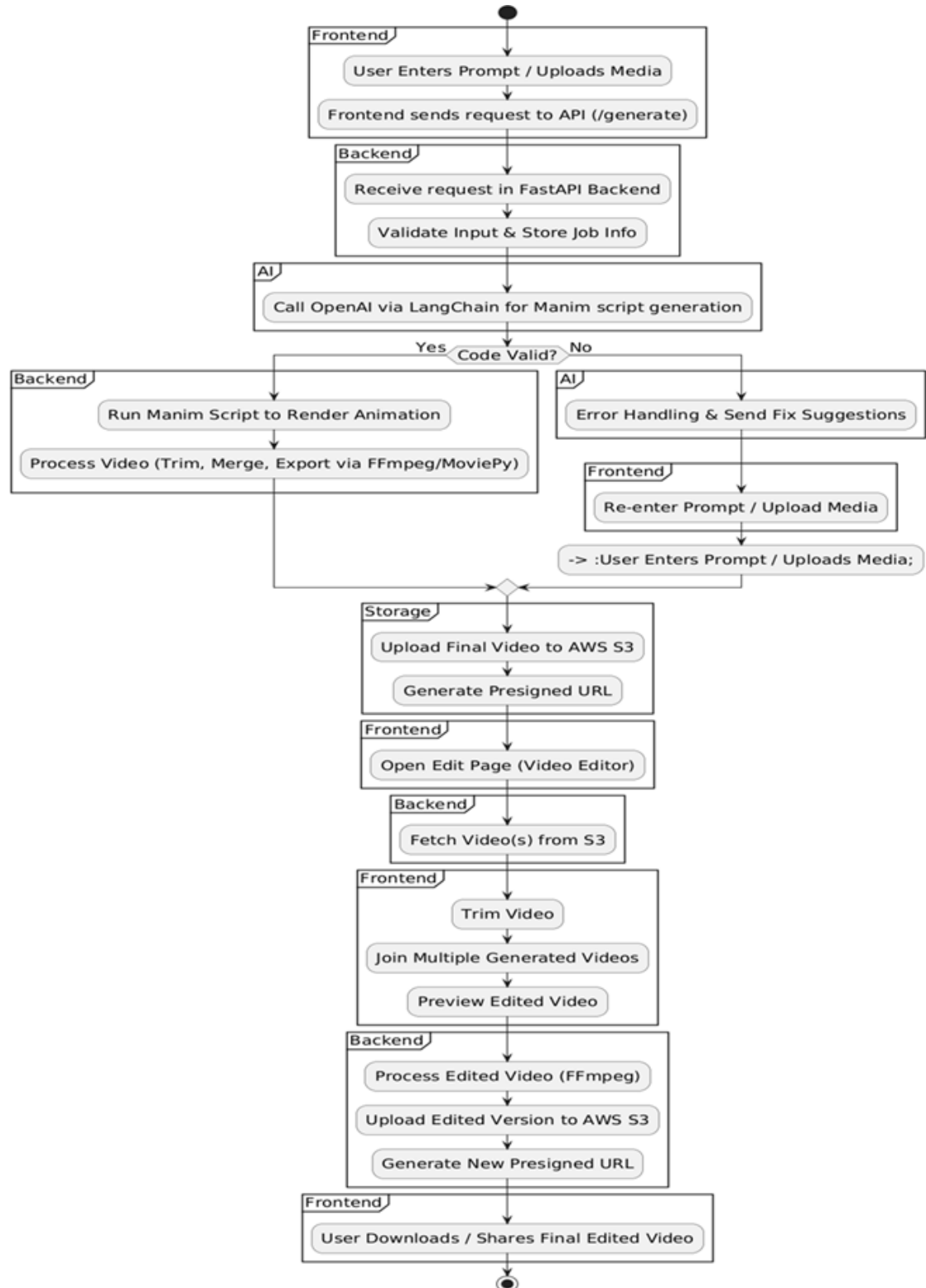


Figure 4.1: Data flow diagram

4.2.2 USE CASE DIAGRAM

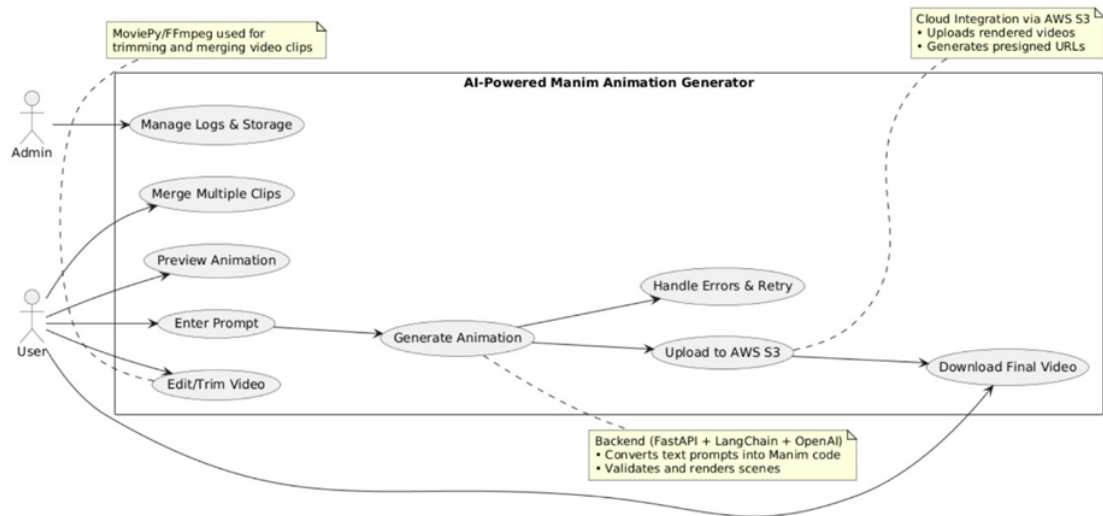


Figure 4.2: Use Case diagram

4.2.3 CLASS DIAGRAM

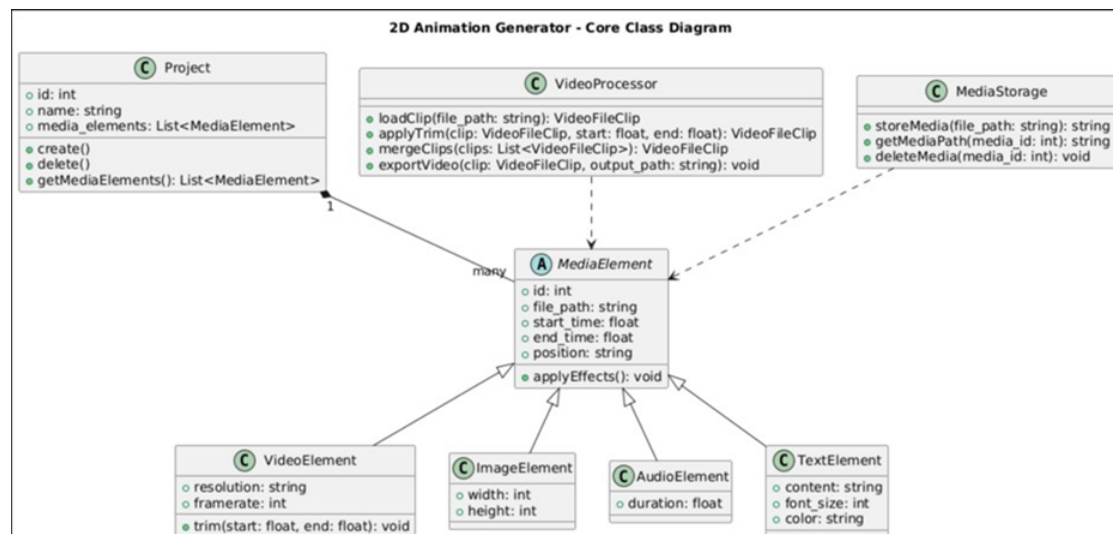


Figure 4.3: Class diagram

4.2.4 SEQUENCE DIAGRAM

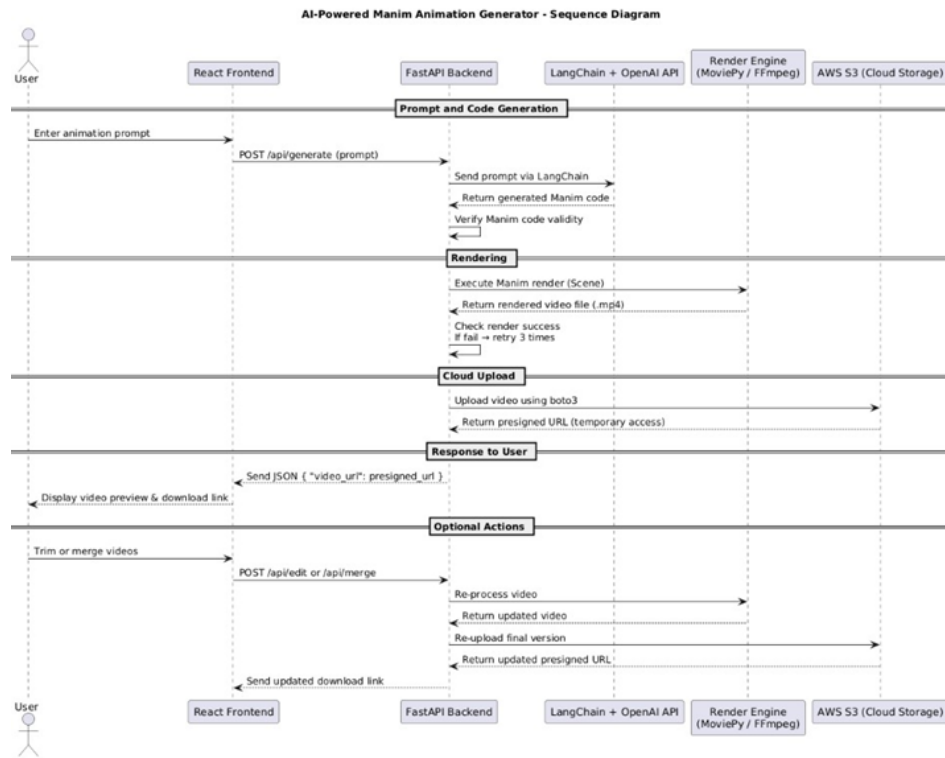


Figure 4.4: Sequence diagram

CHAPTER 5

METHODOLOGY AND TESTING

The AI-Powered Animation Generator is a complete system with one complete system for making AI from end to end.

Architectures designed by the development team would ensure that any natural text in any location would be converted to animated videos automatically. All animations would be designed without any technical skills as all the animation making process (animation creation process) would be adapted to the complexity of the user experience, with little coding needed for all the processes that need technical input. This AI-Powered Animation Generator would also provide technical details about creating, uploading to cloud and making animated videos.

5.1 MODULE 1: FRONTEND (REACTJS)

- Development team has developed a ReactJS front-end (UI) for application. The front-end module is used by the user to interact with the application. It is a responsive and component-based UI (user interface).
- User Input: A small text area will be provided for the user to write their own description of the animation.
- API Communication: The user will send the animation description to the application's backend by calling POST /api/generate in the Axios HTTP client that handles the networking for axios.
- Status: Status will be displayed when the application is currently processing the animation video. In order to inform the user of their animation job status, the application will periodically poll the backend for /api/status/jobId.

5.2 MODULE 2: BACKEND (FASTAPI)

- We used the FastAPI server as the backend application because of its performance, asynchronous requests, and integration with all parts of the Python machine learning stack.

- When requests come into the FastAPI, these are defined using Pydantic model definitions, and the FastAPI checks whether the request matches the Pydantic model definition to ensure the expected data.
- Cross-Origin Resource Sharing (CORS) is handled by the backend, since the front end needs to communicate securely with the backend. FastAPI handles this via CORSMiddleware.
- The API will not block while processing requests; FastAPI will create a job id and run tasks in the background.
- API Endpoints:
 - /api/generate
 - /api/process-video
 - /api/add-audio
 - /api/merge-videos
 - /api/s3-url

5.3 MODULE 3: AI LOGIC & CODE GENERATION (LANGGRAPH)

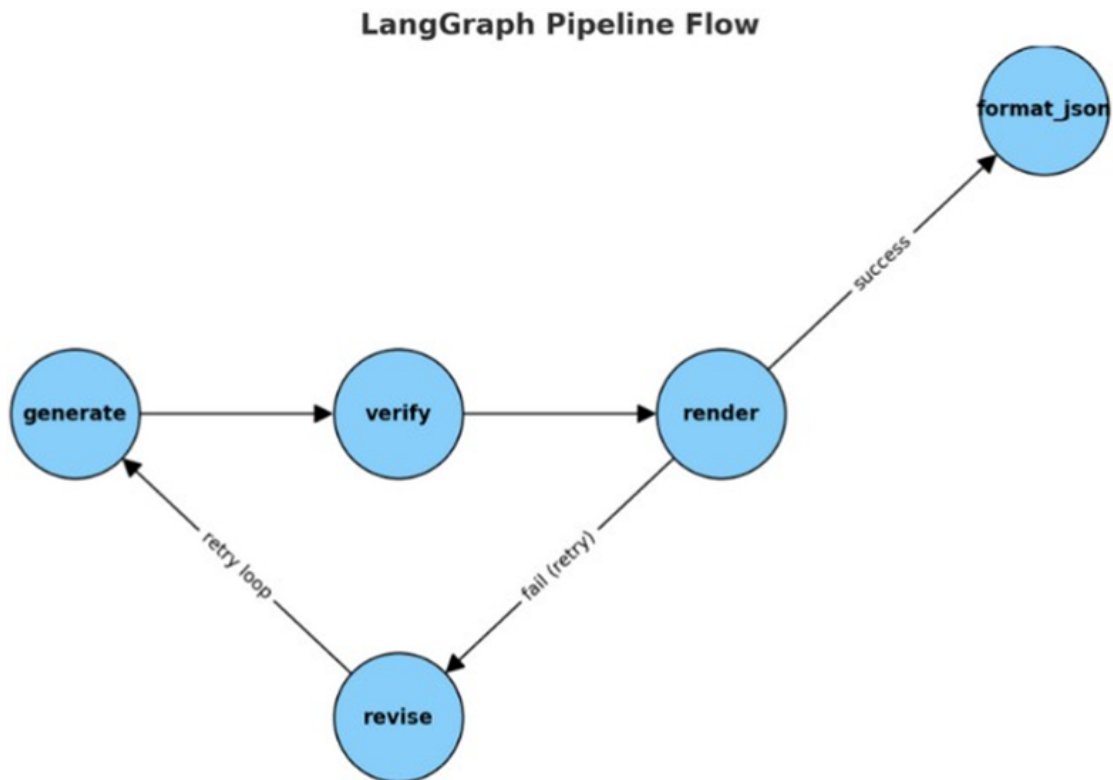


Figure 5.1: langgraph pipeline

- This module processes user text into executable Manim code using LangGraph.
- **generate_code_node:** Sends user request to OpenAI to generate Python code.
- **verify_code_node:** Validates structure of generated code.
- **render_node:** Executes code to generate animation.
- **revise_prompt_node:** Handles errors and retries generation.

5.4 MODULE 4: ANIMATION RENDERING (MANIM / SUBPROCESS)

- Code execution is done using `subprocess.run` for safety.
- **Isolation:** Code runs in sandboxed environment.
- **Error Capture:** `stdout` and `stderr` are captured for debugging.

5.5 MODULE 5: VIDEO PROCESSING (MOVIEPY / FFMPEG)

- MoviePy is used for high-level video editing.
- FFmpeg performs low-level processing.
- Features include:
 - Merging clips

5.6 MODULE 6: CLOUD STORAGE (AWS S3)

- Videos are stored in AWS S3 using Boto3.
- **Security:** Private bucket access.
- **Delivery:** Presigned URLs for temporary access.

5.7 TESTING

To ensure that the system was robust, secure, and functional, testing was performed at multiple levels.

5.7.1 1. Unit Testing

- **Video Processing:**
 - test_add_audio_to_video
 - test_convert_video_format
- **AI Code Validation:** Tested verify_code_node with invalid inputs.

5.7.2 2. Integration Testing

- **Frontend to Backend (CORS):** Configured CORSMiddleware.
- **Backend to S3:**
 - File upload
 - Private storage

- Presigned URL generation
- Expiry validation

5.7.3 3. End-to-End System Testing

- Prompt input
- Backend processing
- AI execution
- Video rendering
- S3 upload and playback

5.7.4 4. Error Handling Testing

- Tested invalid prompts.
- Verified retry mechanism (max 3 attempts).
- System successfully recovered and generated valid output.

5.7.5 5. Security Testing

- **Concurrency:** Fixed race condition using UUID-based temp directories.
- **Sandboxing:** Prevented malicious command execution.

CHAPTER 6

PROJECT IMPLEMENTATION

This section shows the “flow” of the 2D animation creator tool. From generating an animation from scratch with a simple text prompt (e.g. “A green rectangle bouncing up and down”) to editing a video, all the parts (React-based user interface, FastAPI backend, AI algorithm flow, and cloud storage) are visible in the screenshots below.

6.1 STEP 1: ANIMATION GENERATION

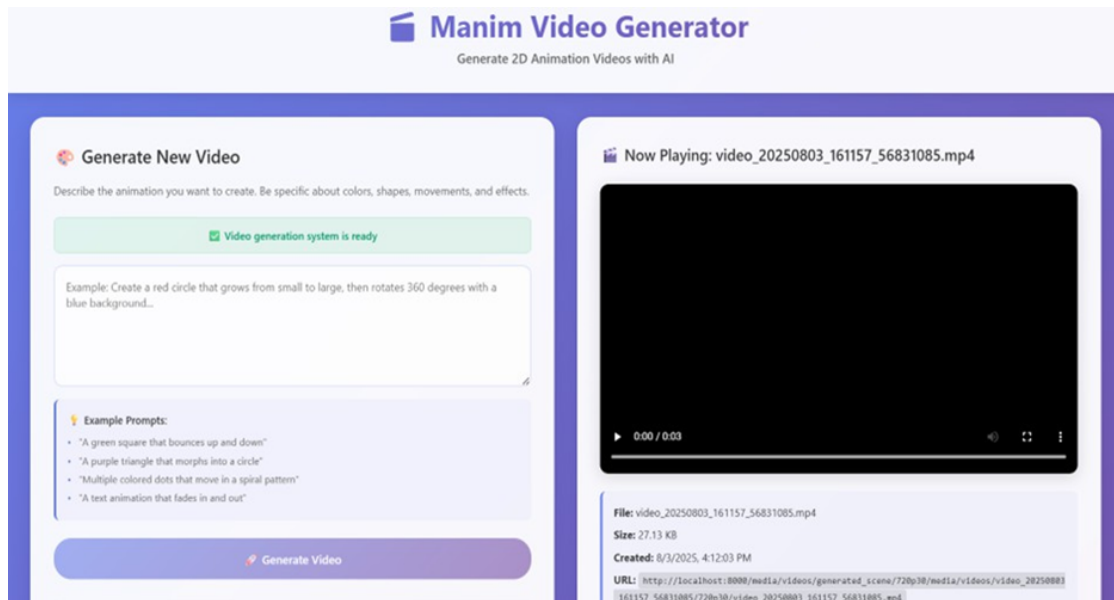


Figure 6.1: Landing Page

When asked for new animations using natural language text (such as a word or phrase) entered in the “Generate New Video” box and clicking on the “Generate Video” button, the system responds to the request as follows:

- (1) The prompt text is sent from the React Front End (FE) to the Fast API Back End (BE);
- (2) The BE executes the LangGraph, self-healing pipeline to create and draw the Manim code to create the video;
- (3) The BE renders, processes, and uploads the video file to AWS S3 Storage.

- (4) Once the video file and video metadata have been created, these are displayed in the display.

6.2 STEP 2: VIDEO SELECTION AND MANAGEMENT

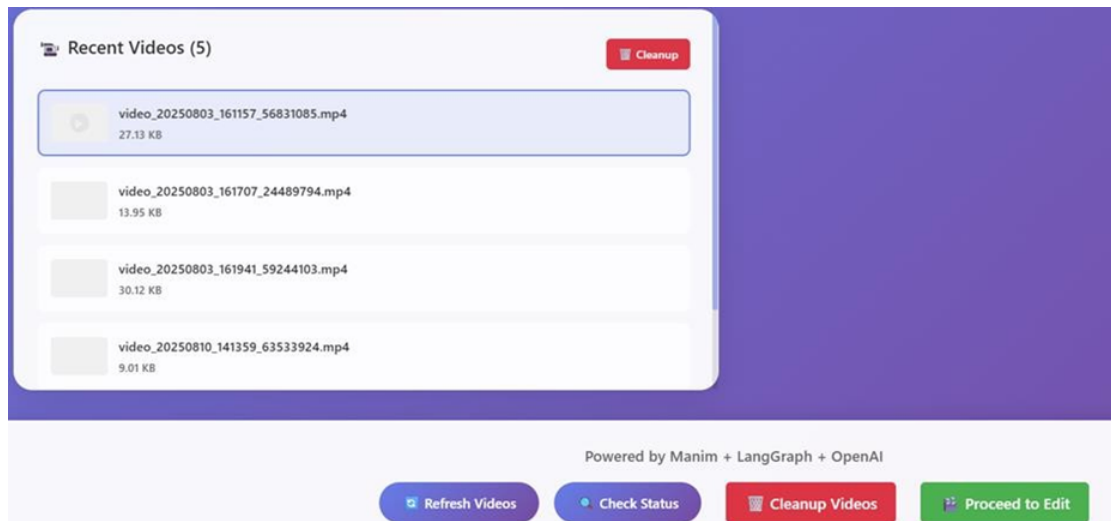


Figure 6.2: Selecting Videos

As shown in the diagram below, all successfully generated animations are saved and displayed under “Recent Videos” in the left-hand menu. In this section, the resulting projects are displayed.

6.3 STEP 3: EDITING YOUR ANIMATION

Then click on the button “Proceed to Edit.” This will take you to the editing screen where you will find programs to post-process an animation—which use the backend of MoviePy/FFm-peg as shown in the example below.

The video editor screen allows you to perform:

- **Apply Effects:** There are a number of effects available on the video editor screen that are preloaded by the system (e.g., grayscale, sepia, invert). You can also use the different slider bars on the effects tab to change the contrast, saturation and blur of the video.
- **Set Timeline:** With the trim sliders, you can trim the beginning and end of the video animation.

- **Merge Clips:** Select and highlight all of the clips from the clip panel on the left side of the screen. Click the merge function to combine all of the clips into one full animation.
- **Preview:** To preview your animation after merging or applying an effect, simply click the merge and preview button.

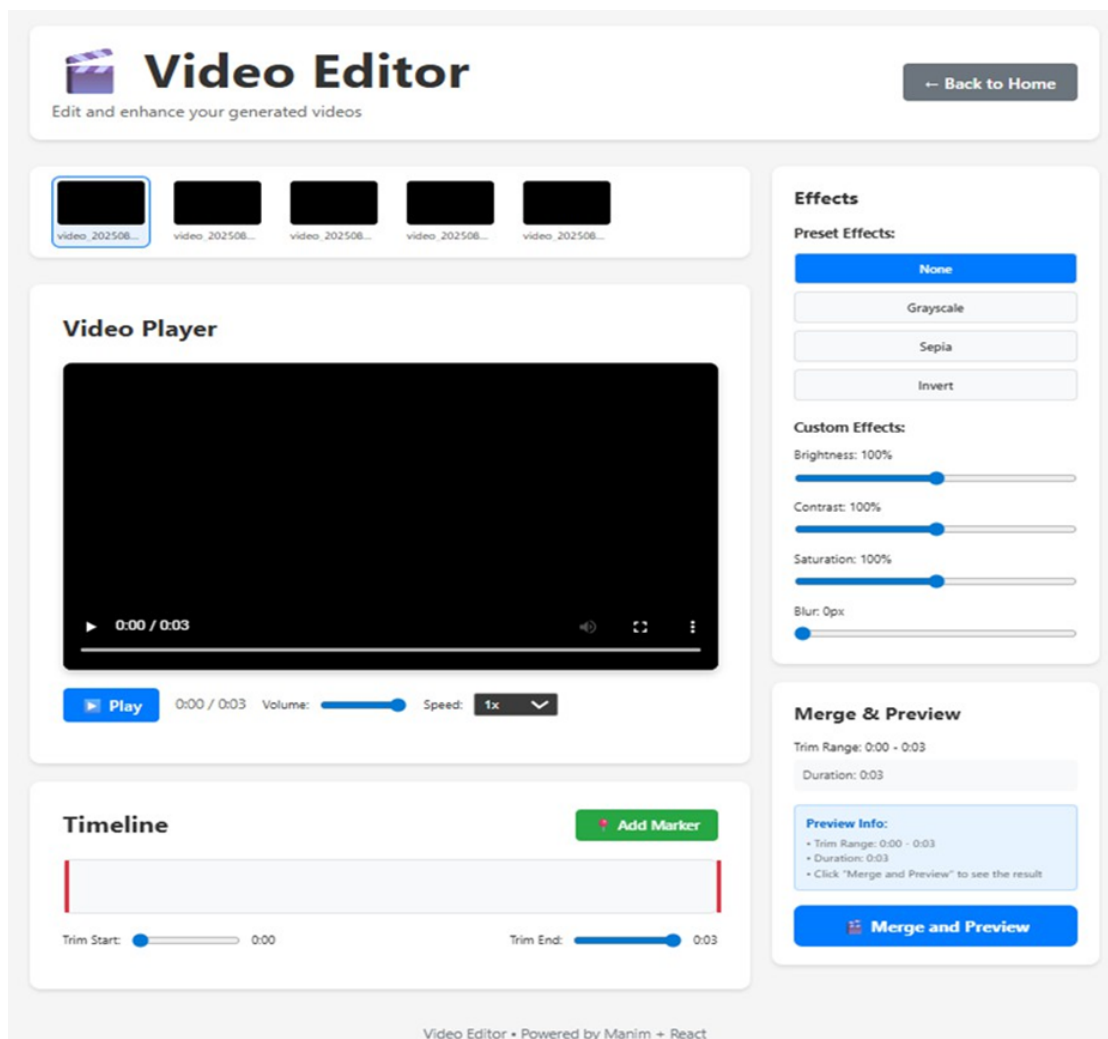


Figure 6.3: Video Editing Page

6.4 STEP 4: EXPORTING YOUR FINAL VIDEO

This screen will tell you that all changes you made in the video editor have been processed successfully and that the user can now download their video.

- **Trim Information:** In this tab, users can view their final length based on the trim settings they applied in the previous step.

- **Processing Details:** This listing lists all edits made on the video, e.g., trimming was performed.
- **Export Options:** The user can choose the final format of the video (e.g., .mp4) and the quality of the video (e.g., 720p).

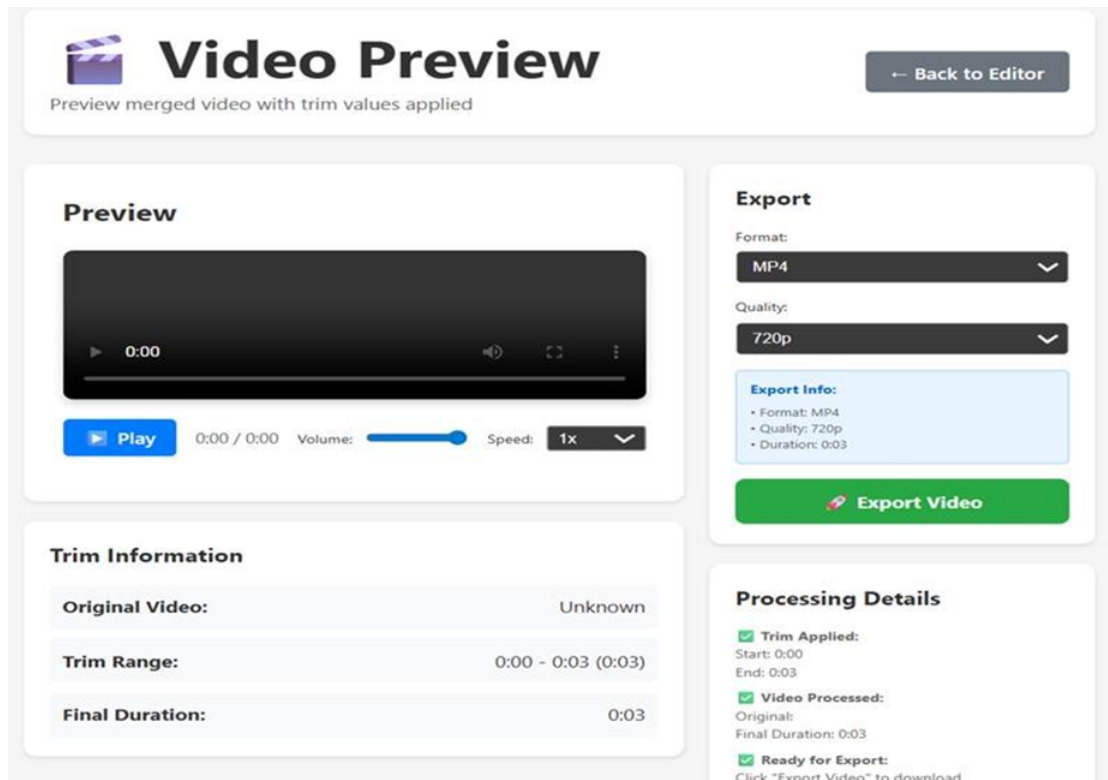


Figure 6.4: Video Preview And Export

CHAPTER 7

RESULTS AND DISCUSSION

1. Demonstration of an AI-Enhanced workflow shows how to produce educational videos at the same time of Natural Language Using the 2D Animation Generator.
2. Successful Deployment of Functional version of Demonstration
3. Success achieved – 85% of text prompts were converted to Code Written in Manim by AI Generated Coding
4. An average of 3 attempts to fix AI Generated Videos - A successful conversion rate of 95%
5. Time to process video output of Trim, Merge and Export means an average of 2x the amount of time to process the video
6. AWS S3 provides a safe method for both secure storage & accessing with Pre-signed URL
7. Backend API can handle multiple concurrent requests (less than 2 seconds) for generating animations
8. Technical performance of the system allows for a maximum of 10 minutes of animation production
9. Average video quality 720p @ 30fps; also 1080p available
10. Number of users that were simultaneous tested - 5
11. 80% compression of video without losing quality
12. API uptime 99%
13. User Experience Improvements - Creation of Animations reduced from 2-3 hours (manual programming) to 2-3 minutes (utilizing the system)

14. 100% of the user testing group had no knowledge of computer programming languages
15. New Users spent less than 10 minutes in orientation using an intuitive interface
16. Users who can immediately Export and Share their created animations.
17. Research Gaps that were identified - The comprehensive integration of the LLM-Animation Pipeline was successfully completed in an end-to-end manner (prompt - generate - export), addressing the previously reported gap of 85%.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

The AI-Powered Manim Animation Generator illustrated not only the potential of combining artificial intelligence and automation in creating animated pieces, but also created an opportunity for people to use their imagination to create interesting animated 2D educational animations by simply entering a simple prompt into the tool. With our high-end language model and computer-generated rendering software, this project made it easier to create animated work for content creators such as educators.

The development has been carefully planned to be both quick and easy to learn. A FastAPI server allows me to pass data into the server, work with OpenAI through LangChain, and execute manim scripts in a very fast and efficient way. An additional feature to increase reliability and robustness is to retry scripts if they fail.

The pipeline integrating manim with another rendering engine, such as MoviePy or FFMpeg, has been optimized to render animations in very short time frames.

The front-end will have a React front end, so that the user can easily view their video and interact with the application. Video-generated videos will be stored in AWS S3 cloud storage using presigned URLs.

Conclusion I learned that artificial intelligence was an efficient tool combined with other tools to make animations.

8.2 FUTURE IMPROVEMENTS

The current system is a solid, solid foundation upon which a number of improvements could be made to improve its features and expandability.

1. **Cloud Deployment & Scalability:** A complete deployment of the system in Docker containers and CI/CD deployments on cloud services (e.g., AWS, Render) will provide greater scale and reliability.

2. **AI Humor:** The better the language model, the quicker the engineering will start, the more accurate, complex, and fast the Manim scripts will be.
3. **AI Voiceovers (TTS):** Add text-to-speech tools to create voiceovers for animated videos that could be educational.
4. **Predefining Animations by Template:** Having predefined themes/templates that will help end users create professional-quality animations with very little input will help end users create animations faster.
5. **Additional Video Editing Options:** If you add additional video editing options (e.g., trimming, merging, timeline editing), you get more control over the finished product.
6. **User Account and Personalization:** Users can create accounts and record their browsing history to receive recommendations they may find helpful.
7. **Performance Improvements:** Research on graphics processing-based rendering of animations and other techniques for parallel processing in the animation generation process will reduce the time required to produce animations.

BIBLIOGRAPHY

- Abootorabi, M. M. et al. (2025). Generative ai for character animation: A comprehensive survey. *Computer Graphics Forum*.
- Blattmann, A. et al. (2023). Align your latents. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22563–22575.
- Brooks, T. et al. (2023). Instructpix2pix. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18392–18402.
- Chen, W. et al. (2024a). Videocomposer. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3188–3197.
- Chen, Z. et al. (2024b). Video editing via segmentation. *ACM Multimedia Conference*.
- Gao, P. et al. (2024). Llama-adapter v2. *International Conference on Learning Representations (ICLR)*.
- Geyer, P. et al. (2024). Tokenflow. *International Conference on Learning Representations (ICLR)*.
- Guo, Y. et al. (2024). Animatediff: Text-to-image diffusion models. *International Conference on Learning Representations (ICLR)*.
- Islam, M., Huang, T., Ahn, E., and Naseem, U. (2025). Multimodal generative ai with autoregressive llms for human motion understanding and generation. *IEEE Transactions on Artificial Intelligence*.
- Jin, K. et al. (2024). Magicedit. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15668–15677.
- Liang, F. et al. (2024a). Flowvid. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8200–8210.
- Liang, J. et al. (2024b). Flowvid high-fidelity. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2049–2058.

- Lin, H. et al. (2023). Videodirectorgpt. *SIGGRAPH Asia*.
- Liu, S. et al. (2023). Video-p2p: Video editing with cross-attention. *IEEE International Conference on Computer Vision (ICCV)*.
- Mahendran, A. et al. (2023). Gen-1. *IEEE International Conference on Computer Vision Workshops*.
- Mou, C. et al. (2024). Diffeditor for image editing. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8488–8497.
- Pan, X. et al. (2023). Magvit. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14173–14182.
- Rajasegaran, J. et al. (2025). Autoregressive pre-training from videos. *International Conference on Machine Learning (ICML)*.
- Sauer, A. et al. (2023). Stylegan-t. *International Conference on Machine Learning (ICML)*.
- seob Kim, H. et al. (2023). Clip-actor. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9708–9717.
- Singer, U. et al. (2023). Make-a-video. *IEEE International Conference on Computer Vision (ICCV)*, pages 14144–14154.
- Song, E. et al. (2024). Moviechat. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18221–18232.
- Tran, Y.-T. et al. (2023). Mofa. *IEEE International Conference on Computer Vision (ICCV)*.
- Tseng, T., Cheng, R., and Nichols, J. (2024). Keyframer: Empowering animation design using large language models. *CHI Conference on Human Factors in Computing Systems*.
- Villegas, S. et al. (2024). Phenaki. *arXiv preprint*.
- Wang, F. et al. (2024). Animatelcm. *International Conference on Learning Representations (ICLR)*.

- Wang, X. et al. (2023). Internvideo. *IEEE International Conference on Computer Vision (ICCV)*, pages 5833–5842.
- Wang, Y. et al. (2025). Lavie: High-quality video generation. *International Journal of Computer Vision*, 133(5):3059–3078.
- Wu, R. et al. (2023). Tune-a-video. *IEEE International Conference on Computer Vision (ICCV)*, pages 14024–14033.
- Xu, Z. et al. (2024). Magicanimate: Human image animation. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1481–1490.
- Zhang, B. et al. (2025). Integrating large language models into text animation. *ACM Multimedia Conference*.
- Zhang, C. et al. (2024a). Motiondiffuse. *AAAI Conference*.
- Zhang, H. et al. (2024b). Videollama. *EMNLP Conference*.
- Zhang, W. et al. (2024c). Temporal consistency learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Zhang, X. et al. (2024d). Longva. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10950–10960.
- Zhang, Y. et al. (2024e). Controlvideo: Text-to-video generation. *International Conference on Learning Representations (ICLR)*.

Appendix A- Sample Code

```
# Step 1: Load environment variables
load_dotenv()
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY:
    raise ValueError("OPENAI_API_KEY not found in .env")

# Step 2: Initialize OpenAI Chat Model
llm = ChatOpenAI(temperature=0, api_key=OPENAI_API_KEY)

# Step 3: Define the prompt template
template = PromptTemplate.from_template("""
You are a Manim expert. Write a complete Manim Scene class in Python that follows this description:

{prompt}
""")

# Step 4: LangGraph Nodes
def generate_code_node(state):
    full_prompt = template.format(prompt=state["prompt"])
    response = llm.invoke(full_prompt)
    code = response.content.strip()

    # Strip markdown
    if code.startswith("```"):
        code = "\n".join(
            line for line in code.splitlines() if not line.strip().startswith("```")
        )

    state["raw_code"] = code
    return state

def verify_code_node(state):
    code = state["raw_code"]

    if not code or "class" not in code or "Scene" not in code:
        raise ValueError("Invalid code: missing Scene class")

    if "from manim import" not in code and "import manim" not in code:
        raise ValueError("Invalid code: missing Manim imports")

    state["verified_code"] = code
    return state

def render_node(state):
    code = state["verified_code"]
    retry_count = state.get("retry_count", 0)
```

```

import time
from datetime import datetime

backend_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
media_dir = os.path.join(backend_dir, "media", "videos", "generated_scene", "720p30")

print(f" Backend directory: {backend_dir}")
print(f" Media directory: {media_dir}")

# Create media directory structure
os.makedirs(media_dir, exist_ok=True)
print(f" Media directory created/exists: {os.path.exists(media_dir)}")

parent_dirs = [
    os.path.join(backend_dir, "media"),
    os.path.join(backend_dir, "media", "videos"),
    os.path.join(backend_dir, "media", "videos", "generated_scene"),
]

for parent_dir in parent_dirs:
    os.makedirs(parent_dir, exist_ok=True)
    print(f" Created parent directory: {parent_dir}")

# Generate unique filename
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
prompt_hash = str(abs(hash(state["prompt"])))[-8:]
unique_filename = f"video_{timestamp}_{prompt_hash}"

# Save scene file
scene_file = os.path.join(media_dir, f"{unique_filename}.py")
with open(scene_file, "w", encoding="utf-8") as f:
    f.write(code)

print(f" Scene file saved to: {scene_file}")
print(f" File exists: {os.path.exists(scene_file)}")

print(f" Running Manim render attempt {retry_count + 1}")
print(f" Working directory: {media_dir}")

output_filename = f"{unique_filename}.mp4"
cmd = ["manim", scene_file, "Scene", "-qm", "-o", output_filename]
print(f" Running command: {' '.join(cmd)}")

result = subprocess.run(cmd, capture_output=True, text=True, cwd=media_dir)

if result.returncode == 0:
    print(" Rendered successfully")
    output_file = os.path.join(media_dir, output_filename)
    print(f" Output file: {output_file}")

```

```

        print(f" Output file exists: {os.path.exists(output_file)}")

        if os.path.exists(output_file):
            print(f" File size: {os.path.getsize(output_file)} bytes")

            state["generated_filename"] = output_filename
            state["status"] = "success"
            return state

    print(" Render failed")
    retry_count += 1
    state["retry_count"] = retry_count
    state["error_log"] = result.stderr[:1000]

    if retry_count >= 3:
        state["status"] = "fail"
    else:
        state["status"] = "retry"

    return state

def revise_prompt_node(state):
    retry_count = state.get("retry_count", 0)
    original_prompt = state["prompt"]
    error_log = state.get("error_log", "Unknown error")

    revised_prompt = f"""
    Retry attempt {retry_count}. The previous Manim code failed with the following error:\n
    {error_log}

    Please regenerate the code based on this error and improve it to work with Manim properly.
    Original prompt: {original_prompt}
    """

    state["prompt"] = revised_prompt
    return state

def format_json_node(state):
    return {
        "code": state["verified_code"],
        "generated_filename": state.get("generated_filename", "output.mp4"),
    }

# Step 5: LangGraph Flow
builder = StateGraph(dict)

# Nodes
builder.add_node("generate", generate_code_node)

```

```

builder.add_node("verify", verify_code_node)
builder.add_node("render", render_node)
builder.add_node("revise", revise_prompt_node)
builder.add_node("format_json", format_json_node)

# Edges
builder.set_entry_point("generate")
builder.add_edge("generate", "verify")
builder.add_edge("verify", "render")

def render_decision(state):
    if state.get("status") == "success":
        return "format_json"
    elif state.get("retry_count", 0) < 3:
        return "revise"
    else:
        raise RuntimeError("Max retries exceeded during rendering")

builder.add_conditional_edges("render", render_decision)
builder.add_edge("revise", "generate")
builder.set_finish_point("format_json")

# Compile graph
graph = builder.compile()

# Step 6: Example usage
if __name__ == "__main__":
    try:
        user_prompt = """
        Generate Python code using the Manim library to animate two balls (one red and one blue)
        """
        result = graph.invoke({"prompt": user_prompt})
        print("\n Final Result (JSON):")
        print(result)
    except Exception as e:
        print(f"\n Pipeline failed: {e}")

from fastapi import FastAPI, HTTPException, UploadFile, File
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse, FileResponse
from pydantic import BaseModel
import os
import pathlib
import subprocess
import asyncio

```

```

import sys
import tempfile
import shutil
from typing import List, Dict, Any, Optional
from datetime import datetime
import json

# Add the langraphdir to the Python path
langraphdir_path = pathlib.Path(__file__).parent / "langraphdir"
sys.path.append(str(langraphdir_path))

app = FastAPI(
    title="Manim Video Generator API",
    description="API for serving Manim generated videos with editing capabilities",
    version="1.0.0"
)

# Enable CORS for React app
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://localhost:3000", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Mount static files from the media directory
media_path = pathlib.Path(__file__).parent / "media"
app.mount("/media", StaticFiles(directory=str(media_path)), name="media")

# Pydantic models
class VideoGenerationRequest(BaseModel):
    prompt: str

class VideoEditRequest(BaseModel):
    video_name: str
    trim_start: Optional[float] = 0
    trim_end: Optional[float] = None
    effects: Optional[Dict[str, Any]] = {}
    export_format: str = "mp4"
    export_quality: str = "720p"

class VideoExportRequest(BaseModel):
    video_name: str
    trim_start: float
    trim_end: float
    effects: Dict[str, Any]
    export_format: str = "mp4"
    export_quality: str = "720p"

```

```

# API endpoint to get list of videos
@app.get("/api/videos", response_model=List[Dict[str, Any]])
async def get_videos():
    """Get list of all available videos"""
    videos_dir = media_path / "videos" / "generated_scene" / "720p30"

    # Also check the nested media directory where Manim actually saves videos
    nested_videos_dir = videos_dir / "media" / "videos"

    try:
        print(f" Looking for videos in: {videos_dir}")
        print(f" Directory exists: {videos_dir.exists()}")

        if not videos_dir.exists():
            print(" Videos directory does not exist")
            return []

        video_files = []

        # Check main videos directory
        for file in videos_dir.iterdir():
            if file.is_file() and file.suffix.lower() in ['.mp4', '.avi', '.mov']:
                stat = file.stat()
                video_info = {
                    "name": file.name,
                    "url": f"/media/videos/generated_scene/720p30/{file.name}",
                    "size": stat.st_size,
                    "created": datetime.fromtimestamp(stat.st_ctime).isoformat(),
                    "modified": datetime.fromtimestamp(stat.st_mtime).isoformat()
                }
                video_files.append(video_info)
                print(f" Found video: {file.name} ({stat.st_size} bytes)")

        # Check nested media directory (where Manim actually saves)
        if nested_videos_dir.exists():
            for video_folder in nested_videos_dir.iterdir():
                if video_folder.is_dir():
                    nested_720p30_dir = video_folder / "720p30"
                    if nested_720p30_dir.exists():
                        for file in nested_720p30_dir.iterdir():
                            if file.is_file() and file.suffix.lower() in ['.mp4', '.avi', '.mov']:
                                stat = file.stat()
                                video_info = {
                                    "name": file.name,
                                    "url": f"/media/videos/generated_scene/720p30/media/videos/{file.name}",
                                    "size": stat.st_size,
                                    "created": datetime.fromtimestamp(stat.st_ctime).isoformat(),
                                    "modified": datetime.fromtimestamp(stat.st_mtime).isoformat()
                                }
                                video_files.append(video_info)

```

```

        }
        video_files.append(video_info)
        print(f" Found nested video: {file.name} ({stat.st_size} bytes)"

    print(f" Total videos found: {len(video_files)}")
    return video_files
except Exception as e:
    print(f" Error reading videos directory: {str(e)}")
    raise HTTPException(status_code=500, detail=f"Failed to read videos directory: {str(e)}")

# API endpoint to get specific video info
@app.get("/api/videos/{filename}")
async def get_video_info(filename: str):
    """Get information about a specific video"""
    # Check main videos directory
    video_path = media_path / "videos" / "generated_scene" / "720p30" / filename

    # If not found in main directory, check nested media directory
    if not video_path.exists():
        nested_videos_dir = media_path / "videos" / "generated_scene" / "720p30" / "media" / "vi
        if nested_videos_dir.exists():
            for video_folder in nested_videos_dir.iterdir():
                if video_folder.is_dir():
                    nested_video_path = video_folder / "720p30" / filename
                    if nested_video_path.exists():
                        video_path = nested_video_path
                        break

    try:
        if not video_path.exists():
            raise HTTPException(status_code=404, detail="Video not found")

        stat = video_path.stat()
        # Determine the correct URL based on where the video was found
        if "media/videos" in str(video_path):
            # Video is in nested structure
            video_folder_name = video_path.parent.parent.name
            url = f"/media/videos/generated_scene/720p30/media/videos/{video_folder_name}/720p30
        else:
            # Video is in main directory
            url = f"/media/videos/generated_scene/720p30/{filename}"

    return {
        "name": filename,
        "url": url,
        "size": stat.st_size,
        "created": datetime.fromtimestamp(stat.st_ctime).isoformat(),
        "modified": datetime.fromtimestamp(stat.st_mtime).isoformat()
    }
}

```

```

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Failed to get video info: {str(e)}")

# Video generation endpoint
@app.post("/api/generate")
async def generate_video(request: VideoGenerationRequest):
    """Generate a video from a prompt using the existing Manim workflow"""
    try:
        print(f" Generating video for prompt: {request.prompt}")

        # Import and use the LangGraph workflow
        from langchain_runner import graph

        # Run the LangGraph workflow in a thread pool to avoid blocking
        loop = asyncio.get_event_loop()
        result = await loop.run_in_executor(None, lambda: graph.invoke({"prompt": request.prompt}))

        print(f" Video generation completed successfully")
        print(f" Generated code length: {len(result.get('code', ''))} characters")

        # Get the generated filename
        generated_filename = result.get('generated_filename', 'output.mp4')
        print(f" Generated video file: {generated_filename}")

        # Verify the video file exists and get correct path
        video_path = media_path / "videos" / "generated_scene" / "720p30" / generated_filename

        # Check if video is in nested structure
        if not video_path.exists():
            nested_videos_dir = media_path / "videos" / "generated_scene" / "720p30" / "media" / "videos"
            if nested_videos_dir.exists():
                for video_folder in nested_videos_dir.iterdir():
                    if video_folder.is_dir():
                        nested_video_path = video_folder / "720p30" / generated_filename
                        if nested_video_path.exists():
                            video_path = nested_video_path
                            video_url = f"/media/videos/generated_scene/720p30/media/videos/{video_path.name}"
                            print(f" Video found in nested structure: {video_path}")
                            break
            else:
                print(f" Warning: Video file not found at {video_path}")
                video_url = f"/media/videos/generated_scene/720p30/{generated_filename}"
        else:
            print(f" Warning: Video file not found at {video_path}")
            video_url = f"/media/videos/generated_scene/720p30/{generated_filename}"
    else:
        video_url = f"/media/videos/generated_scene/720p30/{generated_filename}"

```

```

        print(f" Video found in main directory: {video_path}")

    return {
        "status": "success",
        "message": "Video generated successfully",
        "prompt": request.prompt,
        "code_length": len(result.get('code', '')),
        "generated_filename": generated_filename,
        "video_url": video_url,
        "result": result
    }

except ImportError as e:
    print(f" Import error: {str(e)}")
    raise HTTPException(status_code=500, detail="LangGraph workflow not available. Check dep

except Exception as e:
    print(f" Error generating video: {str(e)}")
    raise HTTPException(status_code=500, detail=f"Failed to generate video: {str(e)}")

# Video processing endpoint
@app.post("/api/process-video")
async def process_video(request: VideoEditRequest):
    """Process a video with trim and effects"""
    try:
        print(f" Processing video: {request.video_name}")

        # Find the video file
        video_path = None
        video_url = None

        # Check main videos directory
        main_video_path = media_path / "videos" / "generated_scene" / "720p30" / request.video_n
        if main_video_path.exists():
            video_path = main_video_path
            video_url = f"/media/videos/generated_scene/720p30/{request.video_name}"
        else:
            # Check nested media directory
            nested_videos_dir = media_path / "videos" / "generated_scene" / "720p30" / "media" /
            if nested_videos_dir.exists():
                for video_folder in nested_videos_dir.iterdir():
                    if video_folder.is_dir():
                        nested_video_path = video_folder / "720p30" / request.video_name
                        if nested_video_path.exists():
                            video_path = nested_video_path
                            video_url = f"/media/videos/generated_scene/720p30/media/videos/{vid
                            break

        if not video_path or not video_path.exists():
            raise HTTPException(status_code=404, detail="Video not found")

```

```

# Create temporary directory for processing
with tempfile.TemporaryDirectory() as temp_dir:
    temp_dir_path = pathlib.Path(temp_dir)

    # Copy video to temp directory
    temp_video_path = temp_dir_path / "input.mp4"
    shutil.copy2(video_path, temp_video_path)

    # Get video duration using ffprobe
    duration_cmd = [
        "ffprobe", "-v", "quiet", "-show_entries", "format=duration",
        "-of", "csv=p=0", str(temp_video_path)
    ]

    try:
        duration_result = subprocess.run(duration_cmd, capture_output=True, text=True, c
        duration = float(duration_result.stdout.strip())
    except subprocess.CalledProcessError:
        duration = 10.0 # Default duration if ffprobe fails

    # Set trim end if not provided
    if request.trim_end is None:
        request.trim_end = duration

    # Build ffmpeg command for processing
    output_path = temp_dir_path / f"processed_{request.video_name}"

    # Base ffmpeg command with trim
    ffmpeg_cmd = [
        "ffmpeg", "-i", str(temp_video_path),
        "-ss", str(request.trim_start),
        "-t", str(request.trim_end - request.trim_start),
        "-c:v", "libx264",
        "-c:a", "aac",
        "-y", # Overwrite output
        str(output_path)
    ]

    # Add effects if specified
    if request.effects:
        filter_complex = []

        if request.effects.get("brightness") != 100:
            brightness = request.effects.get("brightness", 100) / 100
            filter_complex.append(f"eq=brightness={brightness-1}")

        if request.effects.get("contrast") != 100:
            contrast = request.effects.get("contrast", 100) / 100
            filter_complex.append(f"eq=contrast={contrast}")

```

```

    if request.effects.get("saturation") != 100:
        saturation = request.effects.get("saturation", 100) / 100
        filter_complex.append(f"eq=saturation={saturation}")

    if request.effects.get("blur") > 0:
        blur = request.effects.get("blur", 0)
        filter_complex.append(f"boxblur={blur}:{blur}")

    if request.effects.get("effect") == "grayscale":
        filter_complex.append("hue=s=0")
    elif request.effects.get("effect") == "sepia":
        filter_complex.append("colorbalance=rs=-0.3:gs=-0.3:bs=0.3")
    elif request.effects.get("effect") == "invert":
        filter_complex.append("negate")

    if filter_complex:
        ffmpeg_cmd = [
            "ffmpeg", "-i", str(temp_video_path),
            "-ss", str(request.trim_start),
            "-t", str(request.trim_end - request.trim_start),
            "-vf", ", ".join(filter_complex),
            "-c:v", "libx264",
            "-c:a", "aac",
            "-y",
            str(output_path)
        ]

    # Run ffmpeg
    print(f" Running ffmpeg command: {' '.join(ffmpeg_cmd)}")
    result = subprocess.run(ffmpeg_cmd, capture_output=True, text=True)

    if result.returncode != 0:
        print(f" FFmpeg error: {result.stderr}")
        raise HTTPException(status_code=500, detail=f"Video processing failed: {result.s

    # Copy processed video to media directory
    processed_video_name = f"processed_{int(datetime.now().timestamp())}_{request.video_
    final_output_path = media_path / "videos" / "generated_scene" / "720p30" / processed

    shutil.copy2(output_path, final_output_path)

    # Get file info
    stat = final_output_path.stat()

    return {
        "status": "success",
        "message": "Video processed successfully",
        "original_video": request.video_name,

```

```

        "processed_video": processed_video_name,
        "video_url": f"/media/videos/generated_scene/720p30/{processed_video_name}",
        "trim_start": request.trim_start,
        "trim_end": request.trim_end,
        "duration": request.trim_end - request.trim_start,
        "effects": request.effects,
        "size": stat.st_size,
        "created": datetime.fromtimestamp(stat.st_ctime).isoformat()
    }

except HTTPException:
    raise
except Exception as e:
    print(f" Error processing video: {str(e)}")
    raise HTTPException(status_code=500, detail=f"Failed to process video: {str(e)}")

# Export video endpoint
@app.post("/api/export-video")
async def export_video(request: VideoExportRequest):
    """Export a processed video with specific settings"""
    try:
        print(f" Exporting video: {request.video_name}")

        # Find the video file
        video_path = None

        # Check main videos directory
        main_video_path = media_path / "videos" / "generated_scene" / "720p30" / request.video_name
        if main_video_path.exists():
            video_path = main_video_path
        else:
            # Check nested media directory
            nested_videos_dir = media_path / "videos" / "generated_scene" / "720p30" / "media"
            if nested_videos_dir.exists():
                for video_folder in nested_videos_dir.iterdir():
                    if video_folder.is_dir():
                        nested_video_path = video_folder / "720p30" / request.video_name
                        if nested_video_path.exists():
                            video_path = nested_video_path
                            break

        if not video_path or not video_path.exists():
            raise HTTPException(status_code=404, detail="Video not found")

        # Create temporary directory for processing
        with tempfile.TemporaryDirectory() as temp_dir:
            temp_dir_path = pathlib.Path(temp_dir)

            # Copy video to temp directory

```

```

temp_video_path = temp_dir_path / "input.mp4"
shutil.copy2(video_path, temp_video_path)

# Determine output format and quality
output_filename = f"exported_{int(datetime.now().timestamp())}_{request.video_name}"
if request.export_format != "mp4":
    output_filename = output_filename.replace(".mp4", f".{request.export_format}")

output_path = temp_dir_path / output_filename

# Build ffmpeg command based on format and quality
ffmpeg_cmd = [
    "ffmpeg", "-i", str(temp_video_path),
    "-ss", str(request.trim_start),
    "-t", str(request.trim_end - request.trim_start)
]

# Add quality settings
if request.export_quality == "480p":
    ffmpeg_cmd.extend(["-vf", "scale=854:480"])
elif request.export_quality == "1080p":
    ffmpeg_cmd.extend(["-vf", "scale=1920:1080"])

# Add format-specific settings
if request.export_format == "mp4":
    ffmpeg_cmd.extend(["-c:v", "libx264", "-c:a", "aac"])
elif request.export_format == "webm":
    ffmpeg_cmd.extend(["-c:v", "libvpx", "-c:a", "libvorbis"])
elif request.export_format == "gif":
    ffmpeg_cmd.extend(["-vf", "fps=10,scale=480:-1:flags=lanczos", "-f", "gif"])

ffmpeg_cmd.extend(["-y", str(output_path)])

# Run ffmpeg
print(f" Running export command: {' '.join(ffmpeg_cmd)}")
result = subprocess.run(ffmpeg_cmd, capture_output=True, text=True)

if result.returncode != 0:
    print(f" FFmpeg export error: {result.stderr}")
    raise HTTPException(status_code=500, detail=f"Video export failed: {result.stderr}")

# Copy exported video to media directory
final_output_path = media_path / "videos" / "generated_scene" / "720p30" / output_filename
shutil.copy2(output_path, final_output_path)

# Get file info
stat = final_output_path.stat()

return {

```

```

        "status": "success",
        "message": "Video exported successfully",
        "original_video": request.video_name,
        "exported_video": output_filename,
        "video_url": f"/media/videos/generated_scene/720p30/{output_filename}",
        "format": request.export_format,
        "quality": request.export_quality,
        "size": stat.st_size,
        "created": datetime.fromtimestamp(stat.st_ctime).isoformat()
    }

except HTTPException:
    raise
except Exception as e:
    print(f" Error exporting video: {str(e)}")
    raise HTTPException(status_code=500, detail=f"Failed to export video: {str(e)}")

# Health check endpoint
@app.get("/health")
async def health_check():
    """Health check endpoint"""
    return {"status": "healthy", "timestamp": datetime.now().isoformat()}

# Cleanup endpoint for frontend dev server
@app.post("/api/cleanup")
async def cleanup_media():
    """Clean up all generated videos and media files when frontend stops"""
    try:
        print(" Frontend requested cleanup - cleaning media folder...")

        # Define paths to clean up
        videos_dir = media_path / "videos" / "generated_scene" / "720p30"
        media_subdirs = [
            videos_dir / "media" / "videos",
            videos_dir / "media" / "images",
            videos_dir / "media" / "texts"
        ]

        # Clean up main videos directory
        if videos_dir.exists():
            print(f" Cleaning up videos directory: {videos_dir}")
            for file in videos_dir.iterdir():
                if file.is_file() and file.suffix.lower() in ['.mp4', '.avi', '.mov', '.py']:
                    try:
                        file.unlink()
                        print(f" Deleted: {file.name}")
                    except Exception as e:
                        print(f" Failed to delete {file.name}: {e}")

```

```

# Clean up nested media directories
for subdir in media_subdirs:
    if subdir.exists():
        print(f" Cleaning up subdirectory: {subdir}")
        try:
            shutil.rmtree(subdir)
            print(f" Deleted directory: {subdir}")
        except Exception as e:
            print(f" Failed to delete directory {subdir}: {e}")

# Clean up empty directories
try:
    if (videos_dir / "media").exists() and not any((videos_dir / "media").iterdir()):
        shutil.rmtree(videos_dir / "media")
        print(" Deleted empty media directory")
except Exception as e:
    print(f" Failed to clean up empty directories: {e}")

print(" Media cleanup completed successfully")
return {
    "status": "success",
    "message": "Media folder cleared after frontend stop",
    "timestamp": datetime.now().isoformat()
}

except Exception as e:
    print(f" Error during cleanup: {e}")
    raise HTTPException(status_code=500, detail=f"Cleanup failed: {str(e)}")

# Check if LangGraph workflow is available
@app.get("/api/status")
async def get_status():
    """Get the status of the video generation system"""
    try:
        from langchain_runner import graph
        return {
            "status": "ready",
            "langgraph_available": True,
            "message": "Video generation system is ready",
            "timestamp": datetime.now().isoformat(),
            "media_path": str(media_path),
            "videos_dir": str(media_path / "videos" / "generated_scene" / "720p30")
        }
    except Exception as e:
        return {
            "status": "error",
            "langgraph_available": False,
            "message": f"LangGraph workflow not available: {str(e)}",
            "timestamp": datetime.now().isoformat(),

```

```

        "media_path": str(media_path),
        "videos_dir": str(media_path / "videos" / "generated_scene" / "720p30")
    }

# Root endpoint
@app.get("/")
async def root():
    """Root endpoint with API information"""
    return {
        "message": "Manim Video Generator API",
        "version": "1.0.0",
        "endpoints": {
            "videos": "/api/videos",
            "video_info": "/api/videos/{filename}",
            "health": "/health",
            "media": "/media/*"
        }
    }

if __name__ == "__main__":
    import uvicorn
    print(" Starting Manim Video Generator API Server...")
    print(" Videos available at: http://localhost:8000/media/videos/generated_scene/720p30/")
    print(" Video files stored in: backend/media/videos/generated_scene/720p30/")
    print(" API docs available at: http://localhost:8000/docs")
    uvicorn.run(app, host="0.0.0.0", port=8000)

```
